temporary_teamname

박경욱

Compiled on November 2, 2026

Contents

1 Mathematics **2**
- 1.1 Bit Function . . . 2
- 1.2 XOR / Linear Basis . . . 2
- 1.3 Fast Walsh-Hadamard Transform (FWT) 2
- 1.4 Modular Inverse . . . 2
- 1.5 Extended Euclidean Algorithm . . . 2
- 1.6 Chinese Remainder Theorem (CRT) . . . 2
- 1.7 Miller Rabin + Pollard Rho . . . 2
- 1.8 Discrete Log ($O(\sqrt{p})$) . . . 2
- 1.9 Discrete Sqrt ($O(\log^2 p)$) . . . 3
- 1.10 Polynomial . . . 3
- 1.11 Power Sums via NTT / Other Polynomial Tricks . . . 4
- 1.12 Berlekamp Massey . . . 5
- 1.13 Floor Sum . . . 5
- 1.14 Kitamasa ($O(k \log k \log N)$) . . . 5

2 Dynamic Programming **5**
- 2.1 Deque trick . . . 5
- 2.2 Convex Hull Trick (CHT) . . . 5
- 2.3 DnC Optimization . . . 6
- 2.4 Knuth Optimization . . . 6
- 2.5 Slope Trick ($O(n \log n)$) . . . 6
- 2.6 Submask Enumeration . . . 6
- 2.7 Sum Over Subsets (SOS) DP . . . 6
- 2.8 Connection Profile DP . . . 6
- 2.9 Alien's Trick . . . 6

3 Data Structure **7**
- 3.1 Splay Tree . . . 7
- 3.2 EER Tree . . . 8
- 3.3 Link-Cut Tree . . . 8
- 3.4 Li-Chao Tree . . . 9

4 Geometry **9**
- 4.1 CCW . . . 9
- 4.2 Convex Hull . . . 9
- 4.3 3D Convex Hull . . . 9
- 4.4 Rotating Callipers . . . 10
- 4.5 Closest Point on Convex Polygon to a Line 10
- 4.6 Minimum Bounding Rectangle ($O(n)$) . . . 10
- 4.7 Closest Pair of Points . . . 10
- 4.8 KD-Tree . . . 10
- 4.9 2D Minimum Enclosing Circle $O(n^2)$ . . . 11
- 4.10 3D Minimum Enclosing Circle . . . 11
- 4.11 Point in Polygon / Segment-Polygon Intersection . . . 11
- 4.12 Point/Segment Reflection and Rotation . 11
- 4.13 3D Distances (Point-Line, Line-Line, Point-Plane) . . . 11
- 4.14 Cut Polygon by Line ($O(n)$) . . . 12
- 4.15 Polygon Union Area . . . 12
- 4.16 Polygon Centroid . . . 12
- 4.17 Circle-Line Intersection . . . 12
- 4.18 Circle-Circle Intersection . . . 12
- 4.19 Segment-Circle Intersection . . . 12
- 4.20 Half Plane Intersection (HPI) . . . 12
- 4.21 Planar Graph . . . 13
- 4.22 Shamos-Hoey . . . 13
- 4.23 Voronoi Diagram . . . 13

5 Graph **14**
- 5.1 SPFA . . . 14
- 5.2 SCC + 2-SAT . . . 14
- 5.3 BCC . . . 15
- 5.4 Cactus . . . 15
- 5.5 Euler Path/Circuit . . . 15
- 5.6 Articulation Point . . . 15
- 5.7 Articulation Bridge . . . 15
- 5.8 Dominator Tree . . . 15
- 5.9 Offline Dynamic Connectivity . . . 16
- 5.10 Online Dynamic MST . . . 16
- 5.11 kth shortest path (Eppstein's algorithm) . 16

6 Tree **17**
- 6.1 Heavy Light Decomposition (HLD) . . . 17
- 6.2 Centroid Decomposition . . . 17
- 6.3 Tree Compression . . . 17
- 6.4 Tree Isomorphism . . . 17

7 Flow / Matching **17**
- 7.1 Bipartite Matching (Hopcroft-Karp) . . . 17
- 7.2 Matching / Matroid Lemmas . . . 18
- 7.3 Max Flow (Dinic) . . . 18
- 7.4 Min Cost Max Flow (MCMF) - Dijkstra + Potentials . . . 18
- 7.5 Stable Marriage Problem . . . 19
- 7.6 Circulation (Lower Bound Flow) . . . 19
- 7.7 Global Min cut (Stoer-Wagner) . . . 19
- 7.8 Gomory-Hu Tree (All-Pairs Min Cut) . . . 20
- 7.9 General Matching (Blossom) . . . 20
- 7.10 Hungarian Algorithm ($O(n^3)$) . . . 20

8 String **21**
- 8.1 KMP . . . 21
- 8.2 Aho-Corasick . . . 21
- 8.3 Hashing (Rabin Karp) . . . 21
- 8.4 Manacher . . . 22
- 8.5 Suffix Array + LCP ($O(n \log^2 n)$) . . . 22
- 8.6 Suffix Automaton . . . 22
- 8.7 Z Algorithm . . . 22
- 8.8 LCS - Hirschberg ($O(nm)$) . . . 22

9 Miscellaneous **22**
- 9.1 Random Generator . . . 22
- 9.2 Rope . . . 22
- 9.3 PBDS . . . 23
- 9.4 Big Integer . . . 23
- 9.5 Mo's . . . 23
- 9.6 CDQ ($O(n \log n)$) . . . 23
- 9.7 Faker's Algorithm ($O(n \log n)$) . . . 23
- 9.8 Matroid Intersection . . . 23
- 9.9 Primes and Divisors Number . . . 24
- 9.10 Lemma . . . 24

10 Template **25**

1 Mathematics

1.1 Bit Function

```
int a = __builtin_popcount(x); // x에서 켜진 비트의 수
int a = __builtin_clz(x); // x에서 앞에 0이 몇 개 연속으로 나오는가
int a = __builtin_ctz(x); // x에서 뒤에 0이 몇 개 연속으로 나오는가
```

1.2 XOR / Linear Basis

```
struct XorBasis {
    static const int B = 62;
    ll basis[B] = {0}; int cnt = 0;
    bool insert(ll x) {
        for(int i=B-1; i>=0; i--) if((x>>i)&1) {
            if(!basis[i]) {
                basis[i] = x;
                cnt++;
                return true;
            }
            x ^= basis[i];
        }
        return false;
    }
    ll max_xor(ll x) { // x와 xor 했을 때 만들 수 있는 최댓값
        for(int i=B-1; i>=0; i--) if((x^basis[i]) > x) x ^= basis[i];
        return x;
    }
    ll min_xor(ll x) { // x와 xor 했을 때 만들 수 있는 최솟값
        for(int i=B-1; i>=0; i--) if(basis[i] && (x^basis[i]) < x) x ^= basis[i];
        return x;
    }
    bool can_represent(ll x) {
        for(int i=B-1; i>=0; i--) if((x>>i)&1) {
            if(!basis[i]) return false;
            x ^= basis[i];
        }
        return true;
    }
};
```

1.3 Fast Walsh-Hadamard Transform (FWT)

```
// FWT: OR / AND / XOR subset convolution mod MOD. a.size() must be a power of 2.
// Time: O(n log n)
const ll INV2 = (MOD+1) / 2;

void fwt_or(vl &a, bool invert) {
    int n = a.size();
    for(int len=1; len<n; len<<=1)
        for(int i=0; i<n; i+=len*2)
            for(int j=i; j<i+len; j++) {
                if(!invert) a[j+len] = (a[j+len] + a[j]) % MOD;
                else a[j+len] = (a[j+len] - a[j] + MOD) % MOD;
            }
}

void fwt_and(vl &a, bool invert) {
    int n = a.size();
    for(int len=1; len<n; len<<=1)
        for(int i=0; i<n; i+=len*2)
            for(int j=i; j<i+len; j++) {
                if(!invert) a[j] = (a[j] + a[j+len]) % MOD;
                else a[j] = (a[j] - a[j+len] + MOD) % MOD;
            }
}

void fwt_xor(vl &a, bool invert) {
    int n = a.size();
    for(int len=1; len<n; len<<=1)
        for(int i=0; i<n; i+=len*2)
            for(int j=i; j<i+len; j++) {
                ll u = a[j], v = a[j+len];
                a[j] = (u + v) % MOD;
                a[j+len] = (u - v + MOD) % MOD;
                if(invert) { a[j] = a[j] * INV2 % MOD; a[j+len] = a[j+len] * INV2 %
                MOD; }
            }
}

// usage: zero-pad a, b to the same power-of-2 length, then e.g. for XOR convolution:
// fwt_xor(a, false); fwt_xor(b, false);
// for(int i=0; i<n; i++) a[i] = a[i]*b[i] % MOD; fwt_xor(a, true);
```

1.4 Modular Inverse

```
fact[0] = fact_inv[0] = 1;
fact[1] = inv[1] = fact_inv[1] = 1;
for(int i=2; i<3005; i++) {
    fact[i] = fact[i-1] * i % MOD;
    inv[i] = (MOD - (MOD / i) * inv[MOD % i] % MOD) % MOD;
    fact_inv[i] = fact_inv[i-1] * inv[i] % MOD;
}
```

1.5 Extended Euclidean Algorithm

```
// ax + by = gcd(a, b);
ll gcd(ll a, ll b, ll &x, ll &y) {
    if(b == 0) {
        x = 1; y = 0;
        return a;
    }
    ll x1, y1;
    ll d = gcd(b, a%b, x1, y1);
    x = y1; y = x1 - y1 * (a/b);
    return d;
}
```

1.6 Chinese Remainder Theorem (CRT)

```
ll gcd(ll a, ll b, ll &x, ll &y) {
    if(b == 0) {
        x = 1; y = 0;
        return a;
    }
    ll x1, y1;
    ll d = gcd(b, a%b, x1, y1);
    x = y1; y = x1 - y1 * (a/b);
    return d;
}

ll find_rem(i128 a, i128 b) {
    return a%b >= 0 ? a%b : a%b + b;
}

pll crt(ll a, ll b, ll r1, ll r2) { // r1 (mod a) & r2 (mod b)
    ll x0, y0;
    ll g = gcd(a, b, x0, y0);
    if((r2 - r1) % g != 0) return {0, 0};
    i128 mod = (i128) a / g * b;
    i128 rem = find_rem(((i128) r1 * (b/g) % mod) * y0, mod) + find_rem(((i128) r2 *
    (a/g) % mod) * x0, mod);
    rem = (rem + mod) % mod;
    return {mod, rem};
}
```

1.7 Miller Rabin + Pollard Rho

```
// Time: is_prime O(log^2 n); pollard_rho expected O(n^(1/4) log n);
// factorize expected O(n^(1/4) log n) per factor
inline ull addmod(ull a, ull b, ull p) { return (ull)(((i128)a+b)%p); }
inline ull mulmod(ull a, ull b, ull p) { return (ull)((i128)a*b%p); }
inline ull powmod(ull a, ull b, ull p) {
    a %= p;
    ull r = 1;
    while(b){
        if(b & 1) r = mulmod(r, a, p);
        a = mulmod(a, a, p);
        b >>= 1;
    }
    return r;
}

bool miller_rabin(ull a, ull n, ull r) {
    if(a >= n) a %= n;
    if(a <= 1) return true;
    ull d = n>>r, s = powmod(a, d, n);
    if(s == 1 || s == n-1) return true;
    while(--r){
        s = powmod(s, 2, n);
        if(s == 1) return false;
        if(s == n-1) return true;
    }
    return false;
}

bool is_prime(ull n) {
    if(n == 2) return true;
    if(n < 2 || n%2 == 0) return false;
    ull d = n >> 1, r = 1;
    while((d&1)==0) { d>>=1; r++; }

#define T(a) miller_rabin(a##ull, n, r)
    if(n < 4759123141ull) return T(2)&&T(7)&&T(61);
    return T(2)&&T(325)&&T(9375)&&T(28178)&&T(450775)&&T(9780504)&&T(1795265022);
#undef T
}

ull gcd(ull a, ull b) {
    if(!b) return a;
    return gcd(b, a%b);
}

ull pollard_rho(ull n) {
    random_device rd;
    mt19937_64 gen(rd());
    uniform_int_distribution<ull> dist(1, n-1);

    while(true) {
        ull a, b, c, g=1;
        a = b = dist(gen);
        c = dist(gen);

        function<ull(ull)> f = [&](ull x) { return addmod(mulmod(x, x, n), c, n); };

        while(g == 1){
            a = f(a);
            b = f(f(b));
            g = gcd(a > b ? a-b : b-a, n);
        }
        if(g != n) return g;
    }
}

void factorize(ull n, vector<ull> &f){
    if(n == 1) return;
    if(n%2 == 0){
        f.push_back(2);
        factorize(n/2, f);
    }
    else if(is_prime(n)) f.push_back(n);
    else{
        ull p = pollard_rho(n);
        factorize(p, f);
        factorize(n/p, f);
    }
}
```

1.8 Discrete Log ($O(\sqrt{p})$)

```
#include <ext/pb_ds/assoc_container.hpp>

ll pow_mod(ll a, ll b, ll p) {
    a %= p;
    ll r = 1;
    while(b) {
        if(b & 1) r = r * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return r;
```

```
}

// smallest x >= 0 with a^x = b (mod p); p must be prime
ll discrete_log(ll a, ll b, ll p) {
    __gnu_pbds::gp_hash_table<ll, __gnu_pbds::null_type> st;
    ll t = (ll) ceil(sqrt((double) p)), k = 1;
    for(int i=0; i<t; i++) { st.insert(k); k = k * a % p; }

    ll inv = pow_mod(k, p-2, p);
    for(int i=0, k=1; i<t; i++, k=k*inv%p) {
        ll x = b * k % p;
        if(st.find(x) == st.end()) continue;
        for(int j=0, k=1; j<t; j++, k=k*a%p) if(k == x) return i * t + j;
    }
    return -1;
}
```

1.9 Discrete Sqrt ($O(\log^2 p)$)

```
ll pow_mod(ll a, ll b, ll p) {
    a %= p;
    ll r = 1;
    while(b) {
        if(b & 1) r = r * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return r;
}

// solve x^2 = a (mod p); p must be an odd prime; returns -1 if a is a non-residue
ll discrete_sqrt(ll a, ll p) {
    if(a == 0) return 0;
    if(pow_mod(a, (p-1)/2, p) != 1) return -1;
    if(p % 4 == 3) return pow_mod(a, (p+1)/4, p);

    ll s = p - 1, n = 2, r = 0, m;
    while(~s & 1) { r++; s >>= 1; }
    while(pow_mod(n, (p-1)/2, p) != p-1) n++;

    ll x = pow_mod(a, (s+1)/2, p), b = pow_mod(a, s, p), g = pow_mod(n, s, p);
    for(;; r=m) {
        ll t = b;
        for(m=0; m<r && t!=1; m++) t = t * t % p;
        if(!m) return x;
        ll gs = pow_mod(g, 1LL << (r-m-1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

1.10 Polynomial

```
// evalmod (multi-point) is O(n log^2 n);
template<typename T>
struct Poly {
    static T MOD, ROOT, ROOT_INV;

    static void set_mod(T mod) {
        MOD = mod;
        ROOT = find_primitive_root(mod);
        ROOT_INV = powers(ROOT, mod-2);
    }

    static T powers(T a, T b) {
        if(b == 0) return 1;
        T res = powers(a, b/2);
        res = (ll) res * res % MOD;
        if(b%2 == 1) res = (ll) res * a % MOD;
        return res;
    }
    static int find_primitive_root(ll p) {
        if(p <= 1) return -1;
        for(ll i=2; i*i<=p; i++) if(p%i == 0) return -1;

        ll phi = p-1;

        ll tmp = phi; vl factors;
        for(ll i=2; i*i<=tmp; i++) if(tmp%i == 0) {
            factors.push_back(i);
            while(tmp%i == 0) tmp /= i;
        }
        if(tmp > 1) factors.push_back(tmp);

        for(ll res=2; res<=p; res++) {
            bool is_primitive_root = true;
            for(ll factor : factors) if(powers(res, phi/factor) == 1) {
                is_primitive_root = false;
                break;
            }
            if(is_primitive_root) return res;
        }

        return -1;
    }

    vector<T> coef;

    Poly(const vector<T>& c) : coef(c) { normalize(); }
    Poly(T c) : coef(1, c) { normalize(); }
    Poly() : coef{ } { }

    int deg() const { return coef.size(); }
    int size() const { return deg(); }

    T operator [] (int i) const { return i < deg() ? coef[i] : 0; }
    T& operator [] (int i) {
        if(i >= deg()) coef.resize(i+1, 0);
        return coef[i];
    }

    Poly& normalize() {
        while(deg() >= 1 && coef.back() == 0) coef.pop_back();
        return *this;
    }
    Poly& modular() {
        for(auto& x : coef) {
            x %= MOD;
            if(x < 0) x += MOD;
        }
        normalize();
        return *this;
    }
    Poly& resize(int sz) {
        coef.resize(sz, 0);
        return *this;
    }
    Poly reverse() const {
        vector<T> rev(rall(coef));
        return Poly(rev);
    }

    Poly operator + (const Poly& other) const {
        int sz = max(deg(), other.deg());
        vector<T> res(sz, 0);
        for(int i=0; i<sz; i++) res[i] = (*this)[i] + other[i];
        return Poly(res);
    }
    Poly& operator += (const Poly& other) { return *this = *this + other; }
    Poly addmod(const Poly& other) const { return (*this + other).modular(); }

    Poly operator - (const Poly& other) const {
        int sz = max(deg(), other.deg());
        vector<T> res(sz, 0);
        for(int i=0; i<sz; i++) res[i] = (*this)[i] - other[i];
        return Poly(res);
    }
    Poly& operator -= (const Poly& other) { return *this = *this - other; }
    Poly submod(const Poly& other) const { return (*this - other).modular(); }

    // long double FFT: exact only while result values stay well below
    // ~1e12; use mulmod otherwise
    Poly operator * (const Poly& other) const {
        int sz = 1;
        while(sz < deg() + other.deg()) sz <<= 1;

        vector<cpx> fa(sz, 0), fb(sz, 0);
        for(int i=0; i<deg(); i++) fa[i] = coef[i];
        for(int i=0; i<other.deg(); i++) fb[i] = other.coef[i];

        fft(fa, false); fft(fb, false);

        for(int i=0; i<sz; i++) fa[i] *= fb[i];
        fft(fa, true);

        vector<T> res(sz);
        for(int i=0; i<sz; i++) res[i] = round(fa[i].real());
        return Poly(res);
    }
    Poly& operator *= (const Poly& other) { return *this = *this * other; }

    static void fft(vector<cpx>& a, bool invert) {
        int n = a.size();
        for(int i=1, j=0; i<n; i++) {
            int bit = n >> 1;
            while(j & bit) { j -= bit; bit >>= 1; }
            j += bit;
            if(i < j) swap(a[i], a[j]);
        }

        for(int len=2; len<=n; len<<=1) {
            ld ang = 2 * M_PI / len * (invert ? -1 : 1);
            cpx cw(cos(ang), sin(ang));
            for(int i=0; i<n; i+=len) {
                cpx w(1);
                for(int j=0; j<len/2; j++) {
                    cpx u = a[i+j], v = a[i+j+len/2] * w;
                    a[i+j] = u + v;
                    a[i+j+len/2] = u - v;
                    w *= cw;
                }
            }
        }

        if(invert) for(int i=0; i<n; i++) a[i] /= n;
    }

    void ntt(vector<T>& a, bool invert) const {
        int n = a.size();
        for(int i=1, j=0; i<n; i++) {
            int bit = n >> 1;
            while(j & bit) { j -= bit; bit >>= 1; }
            j += bit;
            if(i < j) swap(a[i], a[j]);
        }

        vector<T> roots(n);
        roots[0] = 1;
        T cw = powers(invert ? ROOT_INV : ROOT, (MOD-1)/n);
        for(int i=1; i<n; i++) roots[i] = (ll) roots[i-1] * cw % MOD;

        for(int len=2; len<=n; len<<=1) {
            int step = n / len;
            for(int i=0; i<n; i+=len) {
                for(int j=0; j<len/2; j++) {
                    T u = a[i+j], v = (ll) a[i+j+len/2] * roots[j*step] % MOD;
                    a[i+j] = u+v < MOD ? u+v : u+v-MOD;
                    a[i+j+len/2] = u-v >= 0 ? u-v : u-v+MOD;
                }
            }
        }

        if(invert) {
            T inv_n = powers(n, MOD-2);
            for(auto& x : a) x = (ll) x * inv_n % MOD;
        }
    }
```

```
Poly mulmod(const Poly& other) const {
    int sz = 1;
    while(sz < deg() + other.deg()) sz <<= 1;

    vector<T> a(sz, 0), b(sz, 0);
    for(int i=0; i<deg(); i++) a[i] = coef[i] % MOD;
    for(int i=0; i<other.deg(); i++) b[i] = other.coef[i] % MOD;

    ntt(a, false); ntt(b, false);
    for(int i=0; i<sz; i++) a[i] = (ll) a[i] * b[i] % MOD;
    ntt(a, true);

    return Poly(a);
}

Poly invmod(int high_deg) const {
    assert((*this)[0] != 0);

    Poly b(powers((*this)[0], MOD-2));

    while(b.deg() < high_deg) {
        int m = min(b.deg() * 2, high_deg);

        Poly a = *this; a.resize(m);
        b.resize(m);

        b = b.mulmod(Poly(2).submod(a.mulmod(b)));
        b.resize(m);
    }

    return b.resize(high_deg).normalize();
}

pair<Poly, Poly> divmod(const Poly& other) const {
    if(deg() < other.deg()) return {Poly(0), *this};

    int q_deg = deg() - other.deg() + 1;

    Poly rev_inv = other.reverse().invmod(q_deg);
    Poly q = this->reverse().mulmod(rev_inv);
    q = q.resize(q_deg).reverse().normalize();
    Poly r = this->submod(other.mulmod(q)).normalize();

    return {q, r};
}

Poly diffmod() const {
    if(deg() <= 1) return Poly(0);

    vector<T> res(deg()-1, 0);
    for(int i=1; i<deg(); i++) res[i-1] = (ll) coef[i] * i % MOD;

    return Poly(res);
}

Poly intmod() const {
    vector<T> inv(deg()+1, 0);
    inv[1] = 1;
    for(int i=2; i<=deg(); i++) inv[i] = (MOD - MOD/i) * inv[MOD%i] % MOD;

    vector<T> res(deg()+1, 0);
    for(int i=0; i<deg(); i++) res[i+1] = (ll) coef[i] * inv[i+1] % MOD;

    return Poly(res);
}

Poly logmod(int high_deg) const {
    assert((*this)[0] == 1);

    return diffmod().mulmod(invmod(high_deg)).resize(high_deg -
    1).intmod().resize(high_deg).normalize();
}

Poly expmod(int high_deg) const {
    assert((*this)[0] == 0);

    Poly b(1), b_inv(1);
    while(b.deg() < high_deg) {
        int m = min(b.deg() * 2, high_deg);

        Poly f = *this; f.resize(m);
        b = b.mulmod(Poly(1).addmod(f.submod(b.logmod(m)))).resize(m);
    }
    return b.resize(high_deg).normalize();
}

Poly sqrtmod(int high_deg) const {
    assert((*this)[0] == 1);

    T inv2 = powers(2, MOD-2);
    Poly b(1);
    while(b.deg() < high_deg) {
        int m = min(b.deg()*2, high_deg);

        Poly f = *this; f.resize(m);
        Poly sum = b.addmod(f.mulmod(b.invmod(m))).resize(m);
        for(T &x : sum.coef) x = (ll) x * inv2 % MOD;
        b = sum.resize(m);
    }
    return b.resize(high_deg).normalize();
}

Poly powmod(ll k, int high_deg) const {
    assert((*this)[0] == 1);

    Poly l = logmod(high_deg);
    T kk = ((k % MOD) + MOD) % MOD;
    for(T &x : l.coef) x = (ll) x * kk % MOD;
    return l.resize(high_deg).expmod(high_deg);
}

T evalmod(T x) const {
    T result = 0, xn = 1;
    for(int i=0; i<deg(); i++) {
        result = (result + (ll) xn * coef[i]) % MOD;
        xn = (ll) xn * x % MOD;
    }
    return result;
}

vector<T> evalmod(const vector<T>& x) const {
    int n = x.size();
    vector<Poly> subprod(n*4);

    function<void(int, int, int)> build = [&](int index, int l, int r) {
        if(l == r) {
            subprod[index] = Poly({(MOD - x[l]) % MOD, 1});
            return;
        }
        int mid = (l+r)/2;
        build(index*2, l, mid); build(index*2+1, mid+1, r);
        subprod[index] = subprod[index*2].mulmod(subprod[index*2+1]);
    };
    build(1, 0, n-1);

    vector<T> result(n);
    function<void(int, int, int, const Poly&)> eval = [&](int index, int l, int
    r, const Poly& cur) {
        if(l == r) {
            result[l] = cur.evalmod(x[l]);
            return;
        }
        int mid = (l+r)/2;
        eval(index*2, l, mid, cur.divmod(subprod[index*2]).second);
        eval(index*2+1, mid+1, r, cur.divmod(subprod[index*2+1]).second);
    };
    eval(1, 0, n-1, *this);

    return result;
}

// (x_i, y_i)를 전부 지나는 n차 다항식을 반환
static Poly interpolate(const vector<T>& x, const vector<T>& y) {
    int n = x.size();
    vector<Poly> subprod(n*4);

    function<void(int, int, int)> build = [&](int index, int l, int r) {
        if(l == r) {
            subprod[index] = Poly({(MOD - x[l]) % MOD, 1});
            return;
        }
        int mid = (l+r)/2;
        build(index*2, l, mid); build(index*2+1, mid+1, r);
        subprod[index] = subprod[index*2].mulmod(subprod[index*2+1]);
    };
    build(1, 0, n-1);

    vector<T> denom = subprod[1].diffmod().evalmod(x);
    vector<T> c(n);
    for(int i=0; i<n; i++) c[i] = (ll) y[i] * powers(denom[i], MOD-2) % MOD;

    function<Poly(int, int, int)> solve = [&](int index, int l, int r) -> Poly {
        if(l == r) return Poly(c[l]);
        int mid = (l+r)/2;
        return solve(index*2, l,
        mid).mulmod(subprod[index*2+1]).addmod(solve(index*2+1, mid+1,
        r).mulmod(subprod[index*2]));
    };
    return solve(1, 0, n-1);
}

// P(x)/Q(x)를 거듭제곱수로 전개했을 때, n번째 계수 반환
static T bostan_mori(Poly P, Poly Q, ll n) {
    while(n > 0) {
        Poly Q_neg = Q;
        for(int i=1; i<Q_neg.deg(); i+=2) Q_neg[i] = (MOD - Q_neg[i]) % MOD;
        Poly new_P = P.mulmod(Q_neg), new_Q = Q.mulmod(Q_neg);

        int parity = n & 1;
        vector<T> Pc, Qc;
        for(int i=parity; i<new_P.deg(); i+=2) Pc.push_back(new_P[i]);
        for(int i=0; i<new_Q.deg(); i+=2) Qc.push_back(new_Q[i]);
        P = Poly(Pc); Q = Poly(Qc);
        n >>= 1;
    }
    return (ll) P[0] * powers(Q[0], MOD-2) % MOD;
}
};

template<typename T> T Poly<T>::MOD = 0;
template<typename T> T Poly<T>::ROOT = 0;
template<typename T> T Poly<T>::ROOT_INV = 0;

/*
Poly<ll>::set_mod(998244353);
Poly<ll> a;
for(int i=0; i<n; i++) cin >> a[i];
*/
```

1.11 Power Sums via NTT / Other Polynomial Tricks

```
// S(k,x) = sum_{i=0}^{x} i^k mod p, for every k = 0..n at once
// (needs an NTT-friendly MOD set via Poly<ll>::set_mod)
// Time: O(n log n)
vl power_sums(ll x, int n) {
    ll xp1 = (x+1) % MOD, pw = xp1;
    vl A1(n+1), B1(n+1);
    for(int k=0; k<=n; k++) {
        A1[k] = pw * fact_inv[k+1] % MOD;
        pw = pw * xp1 % MOD;
        B1[k] = fact_inv[k+1];
    }

    Poly<ll> C = Poly<ll>(A1).mulmod(Poly<ll>(B1).invmod(n+1)).resize(n+1);

    vl result(n+1);
    for(int k=0; k<=n; k++) result[k] = C[k] * fact[k] % MOD;
    return result;
}
```

```
// Stirling numbers of the 2nd kind, row n: returns {S(n,0), ..., S(n,n)}
// Time: O(n log n)
vl stirling2_row(int n) {
    vl f(n+1), g(n+1); // f[j] = (-1)^j/j!, g[i] = i^n/i!
    for(int j=0; j<=n; j++) f[j] = (j&1) ? (MOD - fact_inv[j]) % MOD : fact_inv[j];
    for(int i=0; i<=n; i++) g[i] = Poly<ll>::powers(i, n) * fact_inv[i] % MOD;

    return Poly<ll>(f).mulmod(Poly<ll>(g)).resize(n+1).coef;
}
```

1.12 Berlekamp Massey

```
// Time: berlekamp_massey O(n^2); guess_nth_term O(n^2 + k^2 log n)
// (k = recurrence order found);
// get_min_poly/det O(n*(n+|M|)) (|M| = number of nonzero matrix entries)
ll ipow(ll x, ll p){
    ll ret = 1, piv = x;
    while(p){
        if(p & 1) ret = ret * piv % MOD;
        piv = piv * piv % MOD;
        p >>= 1;
    }
    return ret;
}
vi berlekamp_massey(vi x){
    vi ls, cur;
    int lf, lastd;
    for(int i=0; i<x.size(); i++){
        ll t = 0;
        for(int j=0; j<cur.size(); j++){
            t = (t + 1ll * x[i-j-1] * cur[j]) % MOD;
        }
        if((t - x[i]) % MOD == 0) continue;
        if(cur.empty()){
            cur.resize(i+1);
            lf = i;
            lastd = (t - x[i]) % MOD;
            continue;
        }
        ll k = -(x[i] - t) * ipow(lastd, MOD-2) % MOD;
        vi c(i-lf-1);
        c.push_back(k);
        for(auto &j : ls) c.push_back(-j * k % MOD);
        if(c.size() < cur.size()) c.resize(cur.size());
        for(int j=0; j<cur.size(); j++){
            c[j] = (c[j] + cur[j]) % MOD;
        }
        if(i-lf+(int)ls.size()>=(int)cur.size()){
            tie(ls, lf, lastd) = make_tuple(cur, i, (t - x[i]) % MOD);
        }
        cur = c;
    }
    for(auto &i : cur) i = (i%MOD + MOD) % MOD;
    return cur;
}
int get_nth(vi rec, vi dp, ll n){
    int m = rec.size();
    vi s(m), t(m);
    s[0] = 1;
    if(m != 1) t[1] = 1;
    else t[0] = rec[0];
    auto mul = [&rec](vi v, vi w){
        int m = v.size();
        vi t(2 * m);
        for(int j=0; j<m; j++){
            for(int k=0; k<m; k++){
                t[j+k] += 1ll * v[j] * w[k] % MOD;
                if(t[j+k] >= MOD) t[j+k] -= MOD;
            }
        }
        for(int j=2*m-1; j>=m; j--){
            for(int k=1; k<=m; k++){
                t[j-k] += 1ll * t[j] * rec[k-1] % MOD;
                if(t[j-k] >= MOD) t[j-k] -= MOD;
            }
        }
        t.resize(m);
        return t;
    };
    while(n){
        if(n & 1) s = mul(s, t);
        t = mul(t, t);
        n >>= 1;
    }
    ll ret = 0;
    for(int i=0; i<m; i++) ret += 1ll * s[i] * dp[i] % MOD;
    return ret % MOD;
}
int guess_nth_term(vi x, ll n){
    if(n < x.size()) return x[n];
    vi v = berlekamp_massey(x);
    if(v.empty()) return 0;
    return get_nth(v, x, n);
}
struct elem{int x, y, v;}; // A_(x, y) <- v, 0-based. no duplicate please..
vi get_min_poly(int n, vector<elem> M){
    // smallest poly P such that A^i = sum_{j < i} {A^j \times P_j}
    vi rnd1, rnd2;
    random_device rd;
    mt19937 gen(rd());
    auto randint = [&gen](int lb, int ub){
        return uniform_int_distribution<int>(lb, ub)(gen);
    };
    for(int i=0; i<n; i++){
        rnd1.push_back(randint(1, MOD-1));
        rnd2.push_back(randint(1, MOD-1));
    }
    vi gobs;
    for(int i=0; i<n*2+2; i++){
        int tmp = 0;
        for(int j=0; j<n; j++){
            tmp += 1ll * rnd2[j] * rnd1[j] % MOD;
            if(tmp >= MOD) tmp -= MOD;
        }
        gobs.push_back(tmp);
        vi nxt(n);
        for(auto &i : M){
            nxt[i.x] += 1ll * i.v * rnd1[i.y] % MOD;
            if(nxt[i.x] >= MOD) nxt[i.x] -= MOD;
        }
        rnd1 = nxt;
    }
    auto sol = berlekamp_massey(gobs);
    reverse(all(sol));
    return sol;
}
ll det(int n, vector<elem> M){
    vi rnd;
    random_device rd;
    mt19937 gen(rd());
    auto randint = [&gen](int lb, int ub){
        return uniform_int_distribution<int>(lb, ub)(gen);
    };
    for(int i=0; i<n; i++) rnd.push_back(randint(1, MOD-1));
    for(auto &i : M){
        i.v = 1ll * i.v * rnd[i.y] % MOD;
    }
    auto sol = get_min_poly(n, M)[0];
    if(n % 2 == 0) sol = MOD - sol;
    for(auto &i : rnd) sol = 1ll * sol * ipow(i, MOD-2) % MOD;
    return sol;
}
```

1.13 Floor Sum

```
// sum_{i=0}^n [(a*i+b)/c]
ll floor_sum(ll a, ll b, ll c, ll n) {
    if(!a) return b / c * (n+1);

    if(a >= c || b >= c) return n * (n+1) / 2 * (a / c) + (n+1) * (b / c) +
    floor_sum(a%c, b%c, c, n);

    ll m = (a * n + b) / c;
    return m * n - floor_sum(c, c-b-1, a, m-1);
}
```

1.14 Kitamasa ($O(k \log k \log N)$)

```
// N-th term (0-indexed) of dp[i+k] = sum_j c[j]*dp[i+j], via x^N mod f(x),
// f(x) = x^k - sum c[j]x^j
ll kitamasa(vl dp, vl c, ll N) {
    int k = c.size();
    vl f(k+1);
    f[k] = 1;
    for(int i=0; i<k; i++) f[i] = (MOD - c[i]) % MOD;
    Poly<ll> mod_poly(f);

    Poly<ll> result(1), base(vl{0, 1}); // result = x^0 = 1, base = x^1
    while(N) {
        if(N & 1) result = result.mulmod(base).divmod(mod_poly).second;
        base = base.mulmod(base).divmod(mod_poly).second;
        N >>= 1;
    }

    ll ans = 0;
    for(int i=0; i<k; i++) ans = (ans + result[i] * dp[i]) % MOD;
    return ans;
}
```

2 Dynamic Programming

2.1 Deque trick

```
// result[i] = min_{j=i-m+1,...,i} arr[j]
deque<pii> dq;
for(int i=1; i<=n; i++) {
    while(!dq.empty() && dq.front().second < i-m+1) dq.pop_front();
    while(!dq.empty() && dq.back().first >= arr[i]) dq.pop_back();
    dq.push_back({arr[i], i});

    result[i] = dq.front().first;
}
```

2.2 Convex Hull Trick (CHT)

```
struct Line { // y = ax+b
    mutable ll a, b, p;
    bool operator < (const Line& o) const { return a < o.a; }
    bool operator < (ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(p,q) = p/q)
    static const ll inf = LLONG_MAX;
    ll div(ll p, ll q) { // floored division
        return p / q - ((p ^ q) < 0 && p % q);
    }
    bool isect(iterator x, iterator y) {
        if(y == end()) return x->p = inf, 0;
        if(x->a == y->a) x->p = x->b > y->b ? inf : -inf;
        else x->p = div(y->b - x->b, x->a - y->a);
        return x->p >= y->p;
    }
    void add(ll a, ll b) {
        auto z = insert({a, b, 0}), y = z++, x = y;
        while(isect(y, z)) z = erase(z);
        if(x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while((y = x) != begin() && (--x)->p >= y->p) isect(x, erase(y));
    }
    ll query(ll x) { // max
        assert(!empty());
        auto l = *lower_bound(x);
        return l.a * x + l.b;
    }
};
```

2.3 DnC Optimization

```
// dp[i][j] = \min_{k<j}(dp[i-1][k] + cost[k][j]); requires opt(j) to be
// non-decreasing in j
// Time: O(n log n) per row (O(n) rows -> O(n^2 log n) total for the full 2D dp)
ll cost(int a, int b) {
    ll dx = c[a].first - p[b].first, dy = c[a].second - p[b].second;
    return dx < 0 && dy < 0 ? 0 : dx * dy;
}

void DnC(int l, int r, int s, int e) {
    if(l > r) return;
    int mid = (l+r)/2, opt = l;

    for(int i=s; i<=e; i++) if(cost(i, mid) < cost(opt, mid)) opt = i;
    dp[mid] = cost(opt, mid);

    DnC(l, mid-1, s, opt); DnC(mid+1, r, opt, e);
}
```

2.4 Knuth Optimization

```
// dp[i][j] = \min_{i<=k<j}(dp[i][k] + dp[k+1][j] + cost(i, j));
// requires cost to satisfy the quadrangle inequality
// Time: O(n^2), down from the naive O(n^3) interval dp
for(int l=1; l<=n; l++) {
    for(int i=1; i+l-1<=n; i++) {
        if(l == 1) { dp[i][i] = 0; opt[i][i] = i; }
        else {
            int j = i+l-1;
            opt[i][j] = -1;
            for(int k=max(opt[i][j-1], i); k<j && k<=opt[i+1][j]; k++) {
                if(opt[i][j] == -1 || dp[i][j] > dp[i][k] + dp[k+1][j] + cost(i, j))
                {
                    dp[i][j] = dp[i][k] + dp[k+1][j] + cost(i, j);
                    opt[i][j] = k;
                }
            }
        }
    }
}
```

2.5 Slope Trick ($O(n \log n)$)

```
int n; cin >> n;
for(int i=1; i<=n; i++) cin >> arr[i];

priority_queue<ll> pq; pq.push(arr[1]);
ll x_shift = 0, result = 0;
for(int i=2; i<=n; i++) {
    x_shift++;
    ll x_min = pq.top() + x_shift;
    if(arr[i] <= x_min) {
        result += x_min - arr[i];
        pq.pop(); pq.push(arr[i] - x_shift);
    }
    pq.push(arr[i] - x_shift);
}

cout << result << "\n";
```

2.6 Submask Enumeration

```
// Enumerate every submask of 'mask' (including mask itself and 0).
// Time: O(3^n) total over all 2^n masks of an n-bit universe
// (each bit is 0 in mask, or 1-and-in-sub, or 1-and-not-in-sub: 3 choices per bit)
for(int sub = mask; ; sub = (sub-1) & mask) {
    // ... use sub ...
    if(sub == 0) break;
}

// usage example: sum[mask] = sum of val[sub] over every submask sub of mask
ll val[1<<20], sum[1<<20];
void submask_sum(int n) {
    for(int mask=0; mask<(1<<n); mask++) {
        for(int sub = mask; ; sub = (sub-1) & mask) {
            sum[mask] += val[sub];
            if(sub == 0) break;
        }
    }
}
```

2.7 Sum Over Subsets (SOS) DP

```
for(int t=0; t<20; t++) for(int i=0; i<(1<<20); i++) if(i&(1<<t)) {
    dp[i] += dp[i^(1<<t)];
}
```

2.8 Connection Profile DP

```
// Time: O(n*m*B_m) where B_m is the m-th Bell number
// (number of distinct profile states), exponential in m
int n, m;
ll arr[15][15];
map<string, ll> dp[15][15];
ll result = 1e18;

void init() {
    string tmp = "";
    for(int i=1; i<=m; i++) tmp += '0';
    dp[0][m][tmp] = 0;
}

bool isvalid(string state) {
    if(state[0] == '0') return true;
    for(int i=1; i<m; i++) if(state[i] == state[0]) return true;
    return false;
}

string norm(string state) {
    int num[10] = {0}; int cnt = 0;
    for(int i=0; i<m; i++) {
        if(state[i] == '0') continue;
        if(!num[state[i]-'0']) num[state[i]-'0'] = ++cnt;
        state[i] = (char) ('0' + num[state[i]-'0']);
    }
    return state;
}

string merge(int x, int y, string state) {
    if(y == 1) {
        if(state[0] == '0') state += '9';
        else state += state[0];
    }
    else {
        if(state[0] == '0' && state[m-1] == '0') state += '9';
        else if(state[0] == '0') state += state[m-1];
        else if(state[m-1] == '0') state += state[0];
        else {
            state += state[0];
            for(int i=1; i<m; i++) if(state[i] == state[m-1]) state[i] = state[0];
        }
    }
    state.erase(state.begin());
    return norm(state);
}

void update(int x, int y, string state, ll val) {
    if(!dp[x][y].count(state)) dp[x][y][state] = val;
    else dp[x][y][state] = min(dp[x][y][state], val);
}

int groupnum(string state) {
    bool check[10] = {false}; int cnt = 0;
    for(int i=0; i<m; i++) {
        if(state[i] == '0' || check[state[i]-'0']) continue;
        check[state[i]-'0'] = true;
        cnt++;
    }
    return cnt;
}

void SolveProblem() {
    cin >> n >> m;
    for(int i=1; i<=n; i++) for(int j=1; j<=m; j++) cin >> arr[i][j];

    init();

    for(int i=1; i<=n; i++) for(int j=1; j<=m; j++) {
        int px = j == 1 ? i-1 : i, py = j == 1 ? m : j-1;
        for(auto it = dp[px][py].begin(); it != dp[px][py].end(); ++it) {
            string curstate = it->first;
            if(isvalid(curstate)) {
                string nxstate = curstate; nxstate.erase(nxstate.begin());
                nxstate += '0';
                update(i, j, nxstate, it->second);
            }
            string nxstate = merge(i, j, curstate);
            update(i, j, nxstate, it->second + arr[i][j]);
        }
    }

    for(int i=1; i<=n; i++) for(int j=1; j<=m; j++) {
        for(auto it = dp[i][j].begin(); it != dp[i][j].end(); ++it) {
            int cnt = groupnum(it->first);
            if(cnt <= 1) result = min(result, it->second);
        }
    }

    cout << result << "\n";
}
```

2.9 Alien's Trick

```
// Split [1..n] into exactly k groups, minimizing the total cost.
// Requires f(k) = (optimal cost with k groups) to be convex in k.
// Binary search a penalty 'lam' charged per group, solve the unconstrained dp,
// then undo the penalty.
// Time: O(n^2 log V) with the naive O(n^2) inner dp below
// (V = search range for lam);
// O(n log n log V) if the inner dp is sped up with CHT / DnC /
// monotone deque instead
// (cost() here also satisfies the quadrangle inequality needed for those).
// usage: set n and sumarr[] (1-indexed prefix sums), then ll ans = alien_trick(k);

// example cost: minimize the sum of (group sum)^2, given prefix sums sumarr[]
ll sumarr[100005];
ll cost(int l, int r) {
    ll s = sumarr[r] - sumarr[l-1];
    return s * s;
}

ll dp[100005]; int cnt[100005];

// O(n^2): try every previous split point j for each i
pair<ll, int> dp_penalty(ll lam) {
    dp[0] = 0; cnt[0] = 0;
    for(int i=1; i<=n; i++) {
        dp[i] = 4e18; cnt[i] = 0;
        for(int j=0; j<i; j++) {
            ll cur = dp[j] + cost(j+1, i) + lam;
            // on a tie, keep the smallest number of groups
            if(cur < dp[i] || (cur == dp[i] && cnt[j]+1 < cnt[i])) {
                dp[i] = cur; cnt[i] = cnt[j]+1;
            }
        }
    }
    return {dp[n], cnt[n]};
}

// O(log V) calls to dp_penalty, so O(n^2 log V) total with the O(n^2) dp above
ll alien_trick(int k) {
    ll s = 0, e = 1e15; // e >= max gap f(k-1) - f(k)
    while(s < e) {
        ll mid = (s+e) / 2;
        if(dp_penalty(mid).second <= k) e = mid;
        else s = mid+1;
    }
    return dp_penalty(s).first - s * k;
}
```

3 Data Structure

3.1 Splay Tree

```
struct splay_tree {
    struct node {
        int l, r, p;
        ll key, val, cnt = 1;
        ll leftmax, rightmax, totalmax;
        bool flip_lazy, dummy;
    };

    node tree[500005]; int node_cnt = 0;
    int root = 0;

    int new_node(ll key, ll val, bool isdummy) {
        int cur = ++node_cnt;
        tree[cur].key = key; tree[cur].val = val; tree[cur].dummy = isdummy;
        return cur;
    }

    void update(int cur) {
        int l = tree[cur].l, r = tree[cur].r;
        if(l) update_lazy(l);
        if(r) update_lazy(r);

        tree[cur].cnt = 1;
        if(l) tree[cur].cnt += tree[l].cnt;
        if(r) tree[cur].cnt += tree[r].cnt;

        tree[cur].leftmax = 0;
        if(l) tree[cur].leftmax = tree[l].leftmax;
        if((!l || tree[l].leftmax == tree[l].cnt) && tree[cur].val == 1)
        tree[cur].leftmax = (l ? tree[l].leftmax : 0) + 1 + (r ? tree[r].leftmax :
        0);

        tree[cur].rightmax = 0;
        if(r) tree[cur].rightmax = tree[r].rightmax;
        if((!r || tree[r].rightmax == tree[r].cnt) && tree[cur].val == 1)
        tree[cur].rightmax = (r ? tree[r].rightmax : 0) + 1 + (l ? tree[l].rightmax :
        0);

        tree[cur].totalmax = 0;
        if(l) tree[cur].totalmax = max(tree[cur].totalmax, tree[l].totalmax);
        if(r) tree[cur].totalmax = max(tree[cur].totalmax, tree[r].totalmax);
        if(tree[cur].val == 1) tree[cur].totalmax = max(tree[cur].totalmax, (l ?
        tree[l].rightmax : 0) + 1 + (r ? tree[r].leftmax : 0));
    }
    void update_lazy(int cur) {
        if(!tree[cur].flip_lazy) return;
        swap(tree[cur].l, tree[cur].r);

        int l = tree[cur].l, r = tree[cur].r;
        if(l) {
            tree[l].flip_lazy = !tree[l].flip_lazy;
            swap(tree[l].leftmax, tree[l].rightmax);
        }
        if(r) {
            tree[r].flip_lazy = !tree[r].flip_lazy;
            swap(tree[r].leftmax, tree[r].rightmax);
        }

        tree[cur].flip_lazy = false;
    }

    void rotate(int cur) {
        int p = tree[cur].p;
        update_lazy(p); update_lazy(cur);

        if(!p) return;

        int b = 0;
        if(cur == tree[p].l) {
            tree[p].l = b = tree[cur].r;
            tree[cur].r = p;
        }
        else {
            tree[p].r = b = tree[cur].l;
            tree[cur].l = p;
        }

        tree[cur].p = tree[p].p;
        tree[p].p = cur;
        if(b) tree[b].p = p;
        if(!tree[cur].p) root = cur;
        else if(tree[tree[cur].p].l == p) tree[tree[cur].p].l = cur;
        else tree[tree[cur].p].r = cur;

        update(p); update(cur);
    }
    void splay(int cur, int nx = 0) {
        while(tree[cur].p != nx) {
            int p = tree[cur].p;
            int pp = tree[p].p;

            if(pp == nx) {
                rotate(cur);
                break;
            }

            if(pp) {
                if((cur == tree[p].l && p == tree[pp].l) || (cur == tree[p].r && p ==
                tree[pp].r)) rotate(p);
                else rotate(cur);
            }
            rotate(cur);
        }
        if(!nx) root = cur;
    }
    int insert(ll key, ll val, bool isdummy) {
        int cur = root;
        if(cur) {
            while(1) {
                if(tree[cur].key == key) return cur;
                if(tree[cur].key > key) {
                    if(!tree[cur].l) break;
                    cur = tree[cur].l;
                }
                else {
                    if(!tree[cur].r) break;
                    cur = tree[cur].r;
                }
            }
        }

        int x = new_node(key, val, isdummy);
        if(!cur) { root = x; return x; }
        if(tree[cur].key > key) tree[cur].l = x;
        else tree[cur].r = x;
        tree[x].p = cur;
        splay(x);
        return x;
    }
    bool find(ll key) {
        int cur = root;
        if(!cur) return false;

        while(1) {
            if(tree[cur].key == key) break;
            if(tree[cur].key > key) {
                if(!tree[cur].l) break;
                cur = tree[cur].l;
            }
            else {
                if(!tree[cur].r) break;
                cur = tree[cur].r;
            }
        }
        splay(cur);
        return tree[cur].key == key;
    }
    void erase(int cur) {
        splay(cur);

        if(!tree[cur].l) {
            root = tree[cur].r;
            if(root) tree[root].p = 0;
            return;
        }
        if(!tree[cur].r) {
            root = tree[cur].l;
            tree[root].p = 0;
            return;
        }

        int l = tree[cur].l, r = tree[cur].r;
        tree[l].p = 0;

        int x = l;
        while(1) {
            update_lazy(x);
            if(!tree[x].r) break;
            x = tree[x].r;
        }
        // x becomes the root of cur's old left subtree; it has no right child
        splay(x);

        tree[x].r = r; tree[r].p = x;
        update(x);
    }

    int get_idx(int cur) {
        splay(cur);
        update_lazy(cur);
        return (tree[cur].l ? tree[tree[cur].l].cnt : 0) + 1;
    }

    void kth(int idx) {
        int cur = root;
        while(1) {
            update_lazy(cur);
            if(tree[cur].l && tree[tree[cur].l].cnt >= idx) cur = tree[cur].l;
            else {
                if(tree[cur].l) idx -= tree[tree[cur].l].cnt;
                if(--idx == 0) break;
                cur = tree[cur].r;
            }
        }
        splay(cur);
    }

    int segment(int s, int e) {
        kth(e+1);
        int tmp = root;
        kth(s-1);
        splay(tmp, root);
        return tree[tree[root].r].l;
    }

    void flip(int s, int e) {
        int node = segment(s, e);
        tree[node].flip_lazy = !tree[node].flip_lazy;
        swap(tree[node].leftmax, tree[node].rightmax);
        update(tree[root].r);
        update(root);
    }

    void print(int cur = 0) {
        if(!cur) cur = root;
        update_lazy(cur);

        if(tree[cur].l) print(tree[cur].l);
        if(!tree[cur].dummy) cout << tree[cur].val << " ";
        if(tree[cur].r) print(tree[cur].r);
    }
} S;

// CURRENT position among elements not yet erased (positions shift after an erase):
//   "1 s e" - reverse the range [s,e]
//   "2 s e" - print the longest run of 1s within [s,e]
```

```
//   "3 p"   - erase the element currently at position p
//   "4 p"   - print the current position of the element originally
//             inserted at position p
int id[100005]; // id[p] = node index of the element originally inserted at position
p
void SolveProblem() {
    int n, q; cin >> n >> q;
    S.insert(0, 0, true); // left boundary dummy
    for(int i=1; i<=n; i++) {
        ll v; cin >> v;
        id[i] = S.insert(i, v, false);
    }
    S.insert(n+1, 0, true); // right boundary dummy

    for(int i=1; i<=q; i++) {
        int type; cin >> type;
        if(type == 1) {
            int s, e; cin >> s >> e;
            S.flip(s+1, e+1); // +1 skips the left dummy
        }
        else if(type == 2) {
            int s, e; cin >> s >> e;
            int node = S.segment(s+1, e+1);
            cout << S.tree[node].totalmax << "\n";
        }
        else if(type == 3) {
            int p; cin >> p;
            S.kth(p+1);
            S.erase(S.root);
        }
        else {
            int p; cin >> p;
            cout << S.get_idx(id[p]) - 1 << "\n"; // -1 undoes the left-dummy offset
        }
    }
}
```

3.2 EER Tree

```
// Palindromic tree (eertree). s must be 1-indexed. Time: O(n) amortized.
// Number of distinct palindromic substrings = node_cnt - 2.
struct node {
    int len, suffix;
    int child[27];
};

node nodes[100005]; int node_cnt = 0;
int root_neg, root_zero, last;
string s;

int new_node(int len, int suffix) {
    int cur = ++node_cnt;
    nodes[cur].len = len; nodes[cur].suffix = suffix;
    memset(nodes[cur].child, 0, sizeof nodes[cur].child);
    return cur;
}

void init() {
    root_neg = new_node(-1, 0);        // imaginary root, len -1
    nodes[root_neg].suffix = root_neg;
    root_zero = new_node(0, root_neg); // empty root, len 0
    last = root_zero;
}

// add s[idx], returns the node of the longest palindromic suffix
int insert(int idx) {
    int cur = last;
    while(idx-nodes[cur].len-1 < 1 || s[idx-nodes[cur].len-1] != s[idx]) cur =
    nodes[cur].suffix;

    int c = s[idx] - 'a' + 1;
    if(nodes[cur].child[c]) return last = nodes[cur].child[c];

    int nx = new_node(nodes[cur].len + 2, root_zero);
    if(nodes[nx].len > 1) {
        int tmp = nodes[cur].suffix;
        while(idx-nodes[tmp].len-1 < 1 || s[idx-nodes[tmp].len-1] != s[idx]) tmp =
        nodes[tmp].suffix;
        nodes[nx].suffix = nodes[tmp].child[c];
    }
    nodes[cur].child[c] = nx;

    return last = nx;
}

void SolveProblem() {
    cin >> s; s = " " + s;
    init();
    for(int i=1; i<s.size(); i++) insert(i);
}
```

3.3 Link-Cut Tree

```
struct link_cut {
    struct node {
        int l, r, p;
        ll key, val;
        pll mx = {-1e18, -1e18};
        bool flip_lazy;
    };

    node tree[100005]; int sz = 0;

    void init(int _sz) {
        sz = _sz;
        for(int i=1; i<=sz; i++) tree[i].key = i;
    }

    bool is_root(int cur) {
        int p = tree[cur].p;
        return !p || (cur != tree[p].l && cur != tree[p].r);
    }
    bool is_left(int cur) {
        int p = tree[cur].p;
        return p && cur == tree[p].l;
    }

    void update(int cur) {
        tree[cur].mx = {tree[cur].val, tree[cur].key};
        if(tree[cur].l) tree[cur].mx = max(tree[cur].mx, tree[tree[cur].l].mx);
        if(tree[cur].r) tree[cur].mx = max(tree[cur].mx, tree[tree[cur].r].mx);
    }

    void update_lazy(int cur) {
        if(!tree[cur].flip_lazy) return;
        swap(tree[cur].l, tree[cur].r);

        if(tree[cur].l) tree[tree[cur].l].flip_lazy ^= 1;
        if(tree[cur].r) tree[tree[cur].r].flip_lazy ^= 1;
        tree[cur].flip_lazy = false;
    }

    void rotate(int cur) {
        int p = tree[cur].p;
        if(is_left(cur)) {
            if(tree[cur].r) tree[tree[cur].r].p = p;
            tree[p].l = tree[cur].r; tree[cur].r = p;
        }
        else {
            if(tree[cur].l) tree[tree[cur].l].p = p;
            tree[p].r = tree[cur].l; tree[cur].l = p;
        }

        if(!is_root(p)) {
            if(is_left(p)) tree[tree[p].p].l = cur;
            else tree[tree[p].p].r = cur;
        }

        tree[cur].p = tree[p].p; tree[p].p = cur;
        update(p); update(cur);
    }

    void splay(int cur) {
        while(!is_root(cur)) {
            int p = tree[cur].p;
            if(!is_root(p)) update_lazy(tree[p].p);
            update_lazy(p); update_lazy(cur);

            if(!is_root(p)) {
                if(is_left(cur) == is_left(p)) rotate(p);
                else rotate(cur);
            }

            rotate(cur);
        }
        update_lazy(cur);
    }
    void access(int cur) {
        splay(cur); tree[cur].r = 0;
        update(cur);

        for(int y=cur; tree[cur].p; splay(cur)) {
            y = tree[cur].p; splay(y);
            tree[y].r = cur; update(y);
        }
    }

    void link(int p, int c) {
        access(c); access(p);
        tree[c].l = p; tree[p].p = c;
        update(c);
    }

    void cut(int cur) {
        access(cur);
        if(!tree[cur].l) return;
        tree[tree[cur].l].p = 0; tree[cur].l = 0;
        update(cur);
    }
    void cut(int a, int b) {
        if(find_parent(a) == b) cut(a);
        else cut(b);
    }

    int find_root(int cur) {
        access(cur);
        while(tree[cur].l) {
            cur = tree[cur].l;
            update_lazy(cur);
        }
        splay(cur);
        return cur;
    }
    int find_parent(int cur) {
        access(cur);
        if(!tree[cur].l) return 0;
        cur = tree[cur].l; update_lazy(cur);
        while(tree[cur].r) {
            cur = tree[cur].r;
            update_lazy(cur);
        }
        splay(cur);
        return cur;
    }
    int find_lca(int a, int b) {
        access(a); access(b);
        splay(a);
        return tree[a].p ? tree[a].p : a;
    }

    void update_node(int cur, ll val) {
        splay(cur);
        tree[cur].val = val;
        update(cur);
    }

    pll query(int a, int b) {
        int lca = find_lca(a, b);
```

```
        pll mx = {tree[lca].val, tree[lca].key};

        access(a); splay(lca);
        if(tree[lca].r) mx = max(mx, tree[tree[lca].r].mx);

        access(b); splay(lca);
        if(tree[lca].r) mx = max(mx, tree[tree[lca].r].mx);

        return mx;
    }

    void reroot(int cur) {
        access(cur); splay(cur);
        tree[cur].flip_lazy ^= 1;
    }

    bool is_connect(int a, int b) { return find_root(a) == find_root(b); }
} S;

void SolveProblem() {
    int n, q; cin >> n >> q;
    S.init(n);
    for(int i=1; i<=n; i++) {
        ll v; cin >> v;
        S.update_node(i, v);
    }

    for(int i=1; i<=q; i++) {
        int type; cin >> type;
        if(type == 1) {
            int a, b; cin >> a >> b;
            S.reroot(b);
            S.link(a, b);
        }
        else if(type == 2) {
            int a, b; cin >> a >> b;
            S.cut(a, b);
        }
        else if(type == 3) {
            int a; ll v; cin >> a >> v;
            S.update_node(a, v);
        }
        else {
            int a, b; cin >> a >> b;
            if(!S.is_connect(a, b)) cout << -1 << "\n";
            else cout << S.query(a, b).first << "\n";
        }
    }
}
```

3.4 Li-Chao Tree

```
// max query; for min, insert (-a, -b) and negate the answer.
const ll X_MIN = -1e9, X_MAX = 1e9, NEG = -4e18;

struct line { // y = ax + b
    ll a, b;
    line() : a(0), b(NEG) { }
    line(ll _a, ll _b) : a(_a), b(_b) { }
    ll get(ll x) { return a * x + b; }
};

struct node {
    int l, r;
    line mx;
};

node tree[1000005]; int node_cnt = 0;
int root;

void add_line(int index, ll l, ll r, line cur_line) {
    line down = tree[index].mx, up = cur_line;
    if(down.get(l) > up.get(l)) swap(down, up);

    if(up.get(r) >= down.get(r)) {
        tree[index].mx = up;
        return;
    }

    ll mid = (l+r) >> 1;
    if(up.get(mid) > down.get(mid)) {
        tree[index].mx = up;
        if(!tree[index].r) tree[index].r = ++node_cnt;
        add_line(tree[index].r, mid+1, r, down);
    }
    else {
        tree[index].mx = down;
        if(!tree[index].l) tree[index].l = ++node_cnt;
        add_line(tree[index].l, l, mid, up);
    }
}

ll find_mx(int index, ll l, ll r, ll x) {
    if(!index) return NEG;
    ll mid = (l+r) >> 1, mx = tree[index].mx.get(x);
    if(x <= mid) return max(mx, find_mx(tree[index].l, l, mid, x));
    return max(mx, find_mx(tree[index].r, mid+1, r, x));
}

void SolveProblem() {
    root = ++node_cnt;
    int q; cin >> q;
    for(int i=1; i<=q; i++) {
        int type; cin >> type;
        if(type == 1) {
            ll a, b; cin >> a >> b;
            add_line(root, X_MIN, X_MAX, line(a, b));
        }
        else {
            ll x; cin >> x;
            cout << find_mx(root, X_MIN, X_MAX, x) << "\n";
        }
    }
}
```

4 Geometry

4.1 CCW

```
ll ccw(pll a, pll b, pll c) { return (b.first - a.first) * (c.second - a.second) - (c.first - a.first) * (b.second - a.second); }
```

4.2 Convex Hull

```
bool cmp_y(pll a, pll b) {
    if(a.second != b.second) return a.second < b.second;
    return a.first < b.first;
}

pll p0;
bool cmp_ccw(pll a, pll b) {
    ll curccw = ccw(p0, a, b);
    if(curccw != 0) return curccw > 0;
    else if(a.second != b.second) return a.second < b.second;
    else return a.first < b.first;
}

vector<pll> create_convex_hull() {
    sort(p+1, p+n+1, cmp_y);

    p0 = p[1];
    sort(p+2, p+n+1, cmp_ccw);

    vector<pll> convex;
    for(int i=1; i<=n; i++) {
        while(convex.size() >= 2) {
            pll second = convex.back(); convex.pop_back();
            pll first = convex.back();
            if(ccw(first, second, p[i]) > 0) {
                convex.push_back(second);
                break;
            }
        }
        convex.push_back(p[i]);
    }
    return convex;
}
```

4.3 3D Convex Hull

```
// 3D convex hull via the incremental algorithm. Time: O(n^2).
struct point3 { ld x, y, z; };
point3 operator - (point3 a, point3 b) { return {a.x - b.x, a.y - b.y, a.z - b.z}; }
point3 cross3(point3 a, point3 b) { return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y*b.x}; }
ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }

int n;
point3 p[505];
struct face3 { int a, b, c; };

ld vol6(point3 a, point3 b, point3 c, point3 d) {
    return dot3(cross3(b-a, c-a), d-a);
}

face3 make_face(int a, int b, int c, int inside_pt) {
    if(vol6(p[a], p[b], p[c], p[inside_pt]) > 0) return {a, c, b};
    return {a, b, c};
}

// usage: fill p[0..n-1] and set n (n >= 4); vector<face3> hull = convex_hull_3d();
vector<face3> convex_hull_3d() {
    int i0 = 0, i1 = 1;
    while(i1 < n && dot3(p[i1]-p[i0], p[i1]-p[i0]) < 1e-18) i1++;
    int i2 = i1+1;
    while(i2 < n) {
        point3 cr = cross3(p[i1]-p[i0], p[i2]-p[i0]);
        if(dot3(cr, cr) > 1e-18) break;
        i2++;
    }
    int i3 = i2+1;
    while(i3 < n && fabsl(vol6(p[i0], p[i1], p[i2], p[i3])) < 1e-9) i3++;
    if(i1 >= n || i2 >= n || i3 >= n) return {};

    vector<face3> faces;
    faces.push_back(make_face(i0, i1, i2, i3));
    faces.push_back(make_face(i0, i1, i3, i2));
    faces.push_back(make_face(i0, i2, i3, i1));
    faces.push_back(make_face(i1, i2, i3, i0));

    for(int t=0; t<n; t++) {
        if(t == i0 || t == i1 || t == i2 || t == i3) continue;

        vector<bool> vis(faces.size());
        bool any = false;
        for(int i=0; i<faces.size(); i++) {
            vis[i] = vol6(p[faces[i].a], p[faces[i].b], p[faces[i].c], p[t]) > 1e-9;
            any = any || vis[i];
        }
        if(!any) continue;

        map<pii, int> owner;
        for(int i=0; i<faces.size(); i++) {
            owner[{faces[i].a, faces[i].b}] = i;
            owner[{faces[i].b, faces[i].c}] = i;
            owner[{faces[i].c, faces[i].a}] = i;
        }

        vector<face3> added;
        for(int i=0; i<faces.size(); i++) if(vis[i]) {
            int e[3][2] = {{faces[i].a, faces[i].b}, {faces[i].b, faces[i].c},
            {faces[i].c, faces[i].a}};
            for(auto &ed : e) {
                int u = ed[0], v = ed[1];
                auto it = owner.find({v, u});
                if(it == owner.end() || !vis[it->second]) added.push_back({u, v, t});
            }
        }
```

```
        vector<face3> kept;
        for(int i=0; i<faces.size(); i++) if(!vis[i]) kept.push_back(faces[i]);
        for(auto &f : added) kept.push_back(f);
        faces = kept;
    }
    return faces;
}
```

4.4 Rotating Callipers

```
pll operator - (const pll &a, const pll &b) {
    return pll(a.first - b.first, a.second - b.second);
}

ll dist(pll a, pll b) {
    ll dx = a.first - b.first, dy = a.second - b.second;
    return dx*dx + dy*dy;
}

// usage: hull in CCW order;
ll rotating_callipers(vector<pll> &convex, pii &ans) {
    int m = convex.size();
    if(m == 1) {
        ans = {0, 0};
        return 0;
    }

    int idx1 = 0, idx2 = 1; ll maxdist = 0;
    for(int i=0; i<m*2; i++) {
        ll d = dist(convex[idx1], convex[idx2]);
        if(d > maxdist) {
            maxdist = d;
            ans = {idx1, idx2};
        }
        int nx1 = (idx1+1)%m, nx2 = (idx2+1)%m;
        pll diff1 = convex[idx1] - convex[nx1], diff2 = convex[idx2] - convex[nx2];
        if(ccw(diff1, pll(0, 0), diff2) < 0) idx2 = nx2;
        else idx1 = nx1;
    }
    return maxdist;
}
```

4.5 Closest Point on Convex Polygon to a Line

```
// Time: O(log n). hull must be convex, CCW order, size >= 1.
// index of the hull vertex maximizing dot(v, dir)
int extreme_point(vector<pll> &hull, pld dir) {
    int n = hull.size();
    pld dp{-dir.second, dir.first}; // dir rotated 90 deg CCW
    vector<pld> e(n);
    for(int i=0; i<n; i++) e[i] = {(ld)(hull[(i+1)%n].first - hull[i].first),
    (ld)(hull[(i+1)%n].second - hull[i].second)};
    pld e0 = e[0];
    auto half_rel = [&](pld x) { ld c = e0.first * x.second - e0.second * x.first;
    if(c > 0) return 0; if(c < 0) return 1; return e0.first * x.first + e0.second *
    x.second > 0 ? 0 : 1; };
    auto less_rel = [&](pld a, pld b) { int ha = half_rel(a), hb = half_rel(b); if(ha
    != hb) return ha < hb; return a.first * b.second - a.second * b.first > 0; };

    int l = 0, r = n-1;
    while(l <= r) {
        int mid = (l+r)/2;
        if(!less_rel(dp, e[mid])) l = mid+1;
        else r = mid-1;
    }
    return (r+1)%n;
}


pair<int, ld> closest_to_line(vector<pll> &hull, pll a, pll b) {
    pld d{(ld)(b.first - a.first), (ld)(b.second - a.second)};
    pld normal{-d.second, d.first};
    ld dlen = sqrtl(d.first * d.first + d.second * d.second);
    auto side = [&](pll q) { return d.first * (q.second - a.second) - d.second *
    (q.first - a.first); };
    ld s0 = side(hull[0]);
    pld dir = s0 >= 0 ? pld{-normal.first, -normal.second} : normal;
    int idx = extreme_point(hull, dir);
    return {idx, fabsl(side(hull[idx])) / dlen};
}
```

4.6 Minimum Bounding Rectangle ($O(n)$)

```
vector<pld> min_bounding_rectangle(vector<pll> &hull) {
    int n = hull.size();
    auto proj = [&](int idx, ld ux, ld uy) { return hull[idx].first*ux +
    hull[idx].second*uy; };

    ld ux = hull[1].first - hull[0].first, uy = hull[1].second - hull[0].second;
    ld len0 = hypotl(ux, uy); ux /= len0; uy /= len0;
    ld vx = -uy, vy = ux;
    int pr = 0, pt = 0, pl = 0; // max along +u, max along +v, max along -u
    for(int i=1; i<n; i++) {
        if(proj(i, ux, uy) > proj(pr, ux, uy)) pr = i;
        if(proj(i, vx, vy) > proj(pt, vx, vy)) pt = i;
        if(proj(i, -ux, -uy) > proj(pl, -ux, -uy)) pl = i;
    }

    ld best = -1;
    vector<pld> best_corners;
    for(int i=0; i<n; i++) {
        int j = (i+1) % n;
        ld cu = hull[j].first - hull[i].first, cv = hull[j].second - hull[i].second;
        ld len = hypotl(cu, cv); cu /= len; cv /= len;
        ld nx = -cv, ny = cu; // inward normal of this edge

        while(proj((pr+1)%n, cu, cv) > proj(pr, cu, cv) + 1e-12) pr = (pr+1) % n;
        while(proj((pt+1)%n, nx, ny) > proj(pt, nx, ny) + 1e-12) pt = (pt+1) % n;
        while(proj((pl+1)%n, -cu, -cv) > proj(pl, -cu, -cv) + 1e-12) pl = (pl+1) % n;

        ld minp = proj(pl,cu,cv), maxp = proj(pr,cu,cv);
        // minq is hull[i] itself: edge i is a supporting line there
        ld minq = proj(i,nx,ny), maxq = proj(pt,nx,ny);
        ld area = (maxp - minp) * (maxq - minq);
        if(best < 0 || area < best) {
            best = area;
            auto to_xy = [&](ld p, ld q) { return pld(p*cu - q*cv, p*cv + q*cu); };
            best_corners = { to_xy(minp,minq), to_xy(maxp,minq), to_xy(maxp,maxq),
            to_xy(minp,maxq) };
        }
    }
    return best_corners;
}
```

4.7 Closest Pair of Points

```
pll p[100005];
ll sqr(ll x) { return x * x; }

// usage: fill p[1..n] (n >= 2) and sort(p+1, p+n+1) (sorts by x then y), then
ll closest_pair(int n, pii &ans) {
    set<tuple<ll, ll, int>> s; // (y, x, idx) of points
    ll d = LLONG_MAX;
    for(int i=1, l=1; i<=n; i++) {
        ll rad = (ll) sqrtl((ld) d) + 1;
        while(p[i].first - p[l].first > rad) s.erase({p[l].second, p[l].first, l}),
        l++;
        for(auto it=s.lower_bound({p[i].second-rad, LLONG_MIN, 0}); it!=s.end() &&
        get<0>(*it)<=p[i].second+rad; it++) {
            ll nd = sqr(p[i].first - get<1>(*it)) + sqr(p[i].second - get<0>(*it));
            if(nd < d) {
                d = nd;
                ans = {i, get<2>(*it)};
            }
        }
        s.insert({p[i].second, p[i].first, i});
    }
    return d;
}
```

4.8 KD-Tree

```
// 2D KD-Tree (median split, alternating axis).
// Time: build O(n log n); nearest/range query O(sqrt n) average, O(n) worst case.
struct node { pll pt; int lc, rc; };
node tree[100005]; int node_cnt = 0;
pll p[100005];

ll sqr(ll x) { return x * x; }
ll dist2(pll a, pll b) { return sqr(a.first - b.first) + sqr(a.second - b.second); }

// usage: fill p[1..n], then build(1, n, 0)
int build(int s, int e, int depth) {
    if(s > e) return 0;
    int mid = (s+e) >> 1;
    nth_element(p+s, p+mid, p+e+1, [&](pll a, pll b) {
        return depth%2 == 0 ? a.first < b.first : a.second < b.second;
    });
    int cur = ++node_cnt;
    tree[cur].pt = p[mid];
    tree[cur].lc = build(s, mid-1, depth+1);
    tree[cur].rc = build(mid+1, e, depth+1);
    return cur;
}

// nearest point to target other than an exact coordinate match with target;
// call with best={}, bestd=LLONG_MAX
void nearest(int index, int depth, pll target, pll &best, ll &bestd) {
    if(!index) return;
    ll d = dist2(tree[index].pt, target);
    if(tree[index].pt != target && d < bestd) { bestd = d; best = tree[index].pt; }

    ll diff = depth%2 == 0 ? target.first - tree[index].pt.first : target.second -
    tree[index].pt.second;
    int near = diff <= 0 ? tree[index].lc : tree[index].rc;
    int far = diff <= 0 ? tree[index].rc : tree[index].lc;

    nearest(near, depth+1, target, best, bestd);
    if(sqr(diff) < bestd) nearest(far, depth+1, target, best, bestd);
}

// counts points with xl<=x<=xr and yl<=y<=yr
int range_count(int index, int depth, ll xl, ll xr, ll yl, ll yr) {
    if(!index) return 0;
    ll x = tree[index].pt.first, y = tree[index].pt.second;
    int cnt = (x>=xl && x<=xr && y>=yl && y<=yr) ? 1 : 0;

    ll v = depth%2 == 0 ? x : y, lo = depth%2 == 0 ? xl : yl, hi = depth%2 == 0 ? xr
    : yr;
    if(lo <= v) cnt += range_count(tree[index].lc, depth+1, xl, xr, yl, yr);
    if(hi >= v) cnt += range_count(tree[index].rc, depth+1, xl, xr, yl, yr);
    return cnt;
}

// example: build once, then answer repeated nearest-neighbor queries
void solve_nearest_queries() {
    int n; cin >> n;
    for(int i=1; i<=n; i++) cin >> p[i].first >> p[i].second;
    int root = build(1, n, 0);

    int q; cin >> q;
    while(q--) {
        ll qx, qy; cin >> qx >> qy;
        pll best = {}; ll bestd = LLONG_MAX;
        nearest(root, 0, {qx, qy}, best, bestd);
        cout << best.first << " " << best.second << "\n";
    }
}

// example: build once, then answer repeated rectangle range-count queries
void solve_range_queries() {
    int n; cin >> n;
    for(int i=1; i<=n; i++) cin >> p[i].first >> p[i].second;
    int root = build(1, n, 0);

    int q; cin >> q;
    while(q--) {
        ll xl, xr, yl, yr; cin >> xl >> xr >> yl >> yr;
        cout << range_count(root, 0, xl, xr, yl, yr) << "\n";
    }
}
```

4.9 2D Minimum Enclosing Circle $O(n^2)$

```
// Time: expected O(n) (randomized incremental, thanks to shuffling the input order)
struct circle {
    pld c; ld r;
    circle() { }
    circle(pld _c, ld _r) : c(_c), r(_r) { }
    friend bool operator < (const circle &c1, const circle &c2) { return c1.r < c2.r;
    }
};

ld mag(pld p) { return p.first * p.first + p.second * p.second; }
ld dist(pld p1, pld p2) {
    ld dx = p2.first - p1.first, dy = p2.second - p1.second;
    return hypot(dx, dy);
}
pld center(pld p1, pld p2, pld p3) {
    ld cx = -((mag(p2) - mag(p1)) * (p3.second - p2.second) - (mag(p2) - mag(p3)) *
    (p1.second - p2.second)) / (2*(p1.first - p2.first) * (p3.second - p2.second) -
    2*(p3.first - p2.first) * (p1.second - p2.second));
    ld cy = -((mag(p2) - mag(p1)) * (p3.first - p2.first) - (mag(p2) - mag(p3)) *
    (p1.first - p2.first)) / (2*(p1.second - p2.second) * (p3.first - p2.first) -
    2*(p3.second - p2.second) * (p1.first - p2.first));
    return pld(cx, cy);
}
pld center(pld p1, pld p2) {
    return pld((p1.first + p2.first) / 2, (p1.second + p2.second) / 2);
}

circle mec3(pld p1, pld p2, pld p3) {
    pld c01 = center(p1, p2); ld r01 = dist(c01, p1);
    if(dist(c01, p3) <= r01 + 1e-9) return circle(c01, r01);
    pld c02 = center(p1, p3); ld r02 = dist(c02, p1);
    if(dist(c02, p2) <= r02 + 1e-9) return circle(c02, r02);
    pld c12 = center(p2, p3); ld r12 = dist(c12, p2);
    if(dist(c12, p1) <= r12 + 1e-9) return circle(c12, r12);
    pld c = center(p1, p2, p3);
    return circle(c, dist(c, p1));
}

int n;
pld points[1005];

void find_minimum_enclosing_circle() {
    cin >> n;
    for(int i=1; i<=n; i++) cin >> points[i].first >> points[i].second;
    random_device rd;
    mt19937 gen(rd());
    shuffle(points+1, points+n+1, gen);

    circle result(pld(0, 0), 0);
    for(int i=1; i<=n; i++) if(dist(result.c, points[i]) > result.r) {
        result.c = points[i];
        result.r = 0;
        for(int j=1; j<i; j++) if(dist(result.c, points[j]) > result.r) {
            result.c = center(points[i], points[j]);
            result.r = dist(result.c, points[j]);
            for(int k=1; k<j; k++) if(dist(result.c, points[k]) > result.r) {
                result = mec3(points[i], points[j], points[k]);
            }
        }
    }

    cout << result.c.first << " " << result.c.second << "\n";
    cout << result.r << "\n";
}
```

4.10 3D Minimum Enclosing Circle

```
struct point {
    ld x, y, z;
    point() { }
    point(ld _x, ld _y, ld _z) : x(_x), y(_y), z(_z) { }
    friend ld dist(point &p1, point &p2) {
        ld dx = p1.x - p2.x, dy = p1.y - p2.y, dz = p1.z - p2.z;
        return sqrt(dx*dx + dy*dy + dz*dz);
    }
};

int n;
point points[10005];
point result;
ld rat = 1;

void find_minimum_enclosing_circle() {
    cin >> n;
    for(int i=1; i<=n; i++) cin >> points[i].x >> points[i].y >> points[i].z;

    result = point(0, 0, 0);
    for(int i=1; i<=n; i++) {
        result.x += points[i].x;
        result.y += points[i].y;
        result.z += points[i].z;
    }
    result.x /= n; result.y /= n; result.z /= n;

    for(int t=0; t<20000; t++) {
        ld maxdist = 0; int idx = -1;
        for(int i=1; i<=n; i++) {
            if(maxdist < dist(result, points[i])) {
                maxdist = dist(result, points[i]);
                idx = i;
            }
        }
        result.x += (points[idx].x - result.x) * rat;
        result.y += (points[idx].y - result.y) * rat;
        result.z += (points[idx].z - result.z) * rat;
        rat *= 0.999;
    }

    ld radius = 0;
    for(int i=1; i<=n; i++) radius = max(radius, dist(result, points[i]));
    cout << radius << "\n";
}
```

4.11 Point in Polygon / Segment-Polygon Intersection

```
bool on_segment(pll p, pll a, pll b) {
    if(a > b) swap(a, b);
    return a <= p && p <= b;
}

// Returns 1 if q is strictly inside, 0 if on the boundary, -1 if outside.
int point_in_convex(vector<pll> &poly, pll q) {
    int n = poly.size();
    ll c1 = ccw(poly[0], poly[1], q), c2 = ccw(poly[0], poly[n-1], q);
    if(c1 < 0 || c2 > 0) return -1;
    if(c1 == 0) return on_segment(q, poly[0], poly[1]) ? 0 : -1;
    if(c2 == 0) return on_segment(q, poly[0], poly[n-1]) ? 0 : -1;

    int l = 1, r = n-2;
    while(l <= r) {
        int mid = (l+r)/2;
        if(ccw(poly[0], poly[mid], q) >= 0) l = mid+1;
        else r = mid-1;
    }
    int lo = r, hi = r+1;
    ll c = ccw(poly[lo], poly[hi], q);
    if(c > 0) return 1;
    if(c == 0) return 0;
    return -1;
}

// Returns 1 if q is strictly inside, 0 if on the boundary, -1 if outside.
int point_in_polygon(vector<pll> &poly, pll q) {
    int n = poly.size();
    bool inside = false;
    for(int i=0; i<n; i++) {
        pll a = poly[i], b = poly[(i+1)%n];
        if(ccw(a, b, q) == 0 && on_segment(q, a, b)) return 0;
        if((a.second > q.second) != (b.second > q.second)) {
            ll cross = (b.first - a.first) * (q.second - a.second) - (q.first -
            a.first) * (b.second - a.second);
            if((cross > 0) == (b.second > a.second)) inside = !inside;
        }
    }
    return inside ? 1 : -1;
}
```

4.12 Point/Segment Reflection and Rotation

```
// reflect point p across point c (point symmetry)
pld reflect_point(pld p, pld c) { return pld(c.first * 2 - p.first, c.second * 2 -
p.second); }

// reflect point p across the line through a,b (a != b)
pld reflect_line(pld p, pld a, pld b) {
    pld d(b.first - a.first, b.second - a.second);
    ld t = ((p.first - a.first) * d.first + (p.second - a.second) * d.second) /
    (d.first * d.first + d.second * d.second);
    pld foot(a.first + t * d.first, a.second + t * d.second);
    return reflect_point(p, foot);
}

// rotate point p by angle theta (radians, CCW) around center c
pld rotate_point(pld p, pld c, ld theta) {
    ld dx = p.first - c.first, dy = p.second - c.second;
    ld cosT = cosl(theta), sinT = sinl(theta);
    return pld(c.first + dx * cosT - dy * sinT, c.second + dx * sinT + dy * cosT);
}

// usage: apply the point functions above to both endpoints for a segment
pair<pld, pld> reflect_segment_point(pair<pld, pld> seg, pld c) {
    return {reflect_point(seg.first, c), reflect_point(seg.second, c)};
}
pair<pld, pld> reflect_segment_line(pair<pld, pld> seg, pld a, pld b) {
    return {reflect_line(seg.first, a, b), reflect_line(seg.second, a, b)};
}
pair<pld, pld> rotate_segment(pair<pld, pld> seg, pld c, ld theta) {
    return {rotate_point(seg.first, c, theta), rotate_point(seg.second, c, theta)};
}
```

4.13 3D Distances (Point-Line, Line-Line, Point-Plane)

```
struct point3 { ld x, y, z; };
point3 operator-(point3 a, point3 b) { return {a.x - b.x, a.y - b.y, a.z - b.z}; }
point3 cross3(point3 a, point3 b) { return {a.y * b.z - a.z * b.y, a.z * b.x - a.x *
b.z, a.x * b.y - a.y * b.x}; }
ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
ld norm3(point3 a) { return sqrtl(dot3(a, a)); }

// distance from point p to the line through a with direction d
ld point_line_dist_3d(point3 p, point3 a, point3 d) {
    return norm3(cross3(p - a, d)) / norm3(d);
}

// distance between line (a1, dir d1) and line (a2, dir d2);
// handles skew, intersecting, and parallel lines
ld line_line_dist_3d(point3 a1, point3 d1, point3 a2, point3 d2) {
    point3 n = cross3(d1, d2);
    ld nn = norm3(n);
    // parallel (or identical) lines
    if(nn < 1e-9) return point_line_dist_3d(a2, a1, d1);
    return fabsl(dot3(a2-a1, n)) / nn; // skew or intersecting (0 if intersecting)
}

// distance from point p to the plane through a with normal n
ld point_plane_dist_3d(point3 p, point3 a, point3 n) {
    return fabsl(dot3(p-a, n)) / norm3(n);
}
// distance from point p to the plane through a, b, c
ld point_plane_dist_3d(point3 p, point3 a, point3 b, point3 c) {
    return point_plane_dist_3d(p, a, cross3(b-a, c-a));
}
```

4.14 Cut Polygon by Line ($O(n)$)

```
vector<pld> cut_polygon(const vector<pld> &poly, pld s, pld e) {
    vector<pld> res;
    int n = poly.size();
    for(int i=0; i<n; i++) {
        pld cur = poly[i], prv = i ? poly[i-1] : poly.back();
        ld a = ccw(s, e, cur), b = ccw(s, e, prv);
        if((a < 0) != (b < 0)) {
            ld t = a / (a - b);
            res.push_back(pld(cur.first + (prv.first - cur.first) * t, cur.second +
            (prv.second - cur.second) * t));
        }
        if(a >= 0) res.push_back(cur);
    }
    return res;
}
```

4.15 Polygon Union Area

```
// Time: O(N^2), N = total vertex count across all polygons.
ld cross2(pld a, pld b) {
    return a.first * b.second - a.second * b.first;
}
pld operator - (pld a, pld b) { return pld(a.first - b.first, a.second - b.second); }
int sgn(ld x) { return (x > 1e-9) - (x < -1e-9); }
ld rat(pld a, pld b) { return sgn(b.first) ? a.first / b.first : a.second / b.second;
}

ld poly_union_area(vector<vector<pld>> &poly) {
    ld ret = 0;
    int np = poly.size();
    for(int i=0; i<np; i++) {
        int ni = poly[i].size();
        for(int v=0; v<ni; v++) {
            pld A = poly[i][v], B = poly[i][(v+1) % ni];
            vector<pair<ld,int>> segs = {{0, 0}, {1, 0}};
            for(int j=0; j<np; j++) if(i != j) {
                int nj = poly[j].size();
                for(int u=0; u<nj; u++) {
                    pld C = poly[j][u], D = poly[j][(u+1) % nj];
                    int sc = sgn(ccw(A, B, C)), sd = sgn(ccw(A, B, D));
                    if(sc != sd) {
                        ld sa = ccw(C, D, A), sb = ccw(C, D, B);
                        if(min(sc,sd) < 0) segs.push_back({sa / (sa-sb), sc > sd ? 1
                        : -1});
                    } else if(sc == 0 && sd == 0 && j < i) {
                        ld dotv = (B.first - A.first) * (D.first - C.first) +
                        (B.second - A.second) * (D.second - C.second);
                        if(dotv > 0) {
                            segs.push_back({rat(C-A, B-A), 1});
                            segs.push_back({rat(D-A, B-A), -1});
                        }
                    }
                }
            }
            sort(segs.begin(), segs.end());
            for(auto &s : segs) s.first = min(max(s.first, (ld)0), (ld)1);
            ld sum = 0;
            int cnt = segs[0].second;
            for(int k=1; k<(int)segs.size(); k++) {
                if(!cnt) sum += segs[k].first - segs[k-1].first;
                cnt += segs[k].second;
            }
            ret += cross2(A,B) * sum;
        }
    }
    return ret / 2;
}
```

4.16 Polygon Centroid

```
pld polygon_centroid(vector<pld> &poly) {
    ld cx = 0, cy = 0, area = 0;
    int n = poly.size();
    for(int i=0; i<n; i++) {
        pld p = poly[i], q = poly[(i+1) % n];
        ld cross = p.first * q.second - q.first * p.second;
        area += cross;
        cx += (p.first + q.first) * cross;
        cy += (p.second + q.second) * cross;
    }
    area /= 2;
    return pld(cx / (6*area), cy / (6*area));
}
```

4.17 Circle-Line Intersection

```
int circle_line_intersect(pld c, ld r, pld a, pld b, vector<pld> &result) {
    pld d(b.first - a.first, b.second - a.second);
    ld len2 = d.first*d.first + d.second*d.second;
    ld t = ((c.first - a.first) * d.first + (c.second - a.second) * d.second) / len2;
    pld foot(a.first + t * d.first, a.second + t * d.second);
    ld dist2 = (foot.first - c.first) * (foot.first - c.first) + (foot.second -
    c.second) * (foot.second - c.second);
    ld r2 = r*r;
    if(dist2 > r2 + 1e-9) return 0;

    ld len = sqrtl(len2);
    ld ux = d.first / len, uy = d.second / len;
    if(dist2 >= r2 - 1e-9) { // tangent
        result.push_back(foot);
        return 1;
    }

    ld h = sqrtl(r2 - dist2);
    result.push_back(pld(foot.first - ux*h, foot.second - uy*h));
    result.push_back(pld(foot.first + ux*h, foot.second + uy*h));
    return 2;
}
```

4.18 Circle-Circle Intersection

```
int circle_circle_intersect(pld c1, ld r1, pld c2, ld r2, vector<pld> &result) {
    ld dx = c2.first - c1.first, dy = c2.second - c1.second;
    ld d2 = dx*dx + dy*dy, d = sqrtl(d2);
    if(d < 1e-12) return (fabsl(r1-r2) < 1e-9) ? -1 : 0; // same center
    if(d > r1+r2 + 1e-9 || d < fabsl(r1-r2) - 1e-9) return 0;

    ld a = (d2 + r1*r1 - r2*r2) / (d*2);
    ld h2 = r1*r1 - a*a;
    pld foot(c1.first + a*dx/d, c1.second + a*dy/d);
    if(h2 < 1e-9) {
        result.push_back(foot);
        return 1;
    }

    ld h = sqrtl(max((ld)0, h2));
    ld ux = -dy / d, uy = dx / d;
    result.push_back(pld(foot.first - ux*h, foot.second - uy*h));
    result.push_back(pld(foot.first + ux*h, foot.second + uy*h));
    return 2;
}
```

4.19 Segment-Circle Intersection

```
int segment_circle_intersect(pld c, ld r, pld a, pld b, vector<pld> &result) {
    vector<pld> line_pts;
    circle_line_intersect(c, r, a, b, line_pts);
    ld dx = b.first - a.first, dy = b.second - a.second, len2 = dx*dx + dy*dy;
    for(auto &p : line_pts) {
        ld t = ((p.first - a.first) * dx + (p.second - a.second) * dy) / len2;
        if(t >= -1e-9 && t <= 1+1e-9) result.push_back(p);
    }
    return result.size();
}
```

4.20 Half Plane Intersection (HPI)

```
bool z(ld a) { return fabs(a) <= 1e-8; }

// ax+by <= c
struct half_plane {
    ld a, b, c;
    half_plane() { }
    half_plane(ld _a, ld _b, ld _c) : a(_a), b(_b), c(_c) { }
    friend bool operator < (const half_plane &h1, const half_plane &h2) {
        bool flag1 = h1.slope() > pld(0, 0);
        bool flag2 = h2.slope() > pld(0, 0);
        if(flag1 != flag2) return flag1 > flag2;
        ld curccw = ccw(pld(0, 0), h1.slope(), h2.slope());
        return z(curccw) ? h1.c * hypot(h2.a, h2.b) < h2.c * hypot(h1.a, h1.b) :
        curccw > 0;
    }
    friend pld cross(const half_plane &h1, const half_plane &h2) {
        ld d = h1.a * h2.b - h2.a * h1.b;
        ld x = (h1.c * h2.b - h1.b * h2.c) / d;
        ld y = (h1.a * h2.c - h1.c * h2.a) / d;
        return pld(x, y);
    }
    friend bool bad(const half_plane &h1, const half_plane &h2, const half_plane &h3)
    {
        if(ccw(pld(0, 0), h1.slope(), h2.slope()) <= 0) return false;
        pld p1 = cross(h1, h2);
        return h3.a * p1.first + h3.b * p1.second >= h3.c;
    }
    pld slope() const { return pld(a, b); }
};

vector<half_plane> half_planes;

bool hpi(vector<pld> &points) {
    sort(all(half_planes));
    deque<int> dq;
    for(int i=0; i<half_planes.size(); i++) {
        if(!dq.empty() && z(ccw(pld(0, 0), half_planes[dq.back()].slope(),
        half_planes[i].slope()))) continue;
        while(dq.size() >= 2) {
            int second = dq.back(); dq.pop_back();
            int first = dq.back();
            if(!bad(half_planes[first], half_planes[second], half_planes[i])) {
                dq.push_back(second);
                break;
            }
        }
        while(dq.size() >= 2) {
            int first = dq.front(); dq.pop_front();
            int second = dq.front();
            if(!bad(half_planes[i], half_planes[first], half_planes[second])) {
                dq.push_front(first);
                break;
            }
        }
        dq.push_back(i);
    }

    while(dq.size() > 2) {
        int second = dq.back(); dq.pop_back();
        int first = dq.back();
        if(!bad(half_planes[first], half_planes[second], half_planes[dq.front()])) {
            dq.push_back(second);
            break;
        }
    }
    while(dq.size() > 2) {
        int first = dq.front(); dq.pop_front();
        int second = dq.front();
        if(!bad(half_planes[dq.back()], half_planes[first], half_planes[second])) {
            dq.push_front(first);
            break;
        }
    }

    vi v;
    while(!dq.empty()) {
        int cur = dq.front(); dq.pop_front();
```

```
        v.push_back(cur);
    }

    for(int i=0; i<v.size(); i++) {
        int j = (i+1)%v.size();
        if(ccw(pld(0, 0), half_planes[v[i]].slope(), half_planes[v[j]].slope()) <=
        1e-8) return false;
        points.push_back(cross(half_planes[v[i]], half_planes[v[j]]));
    }
    return true;
}
```

4.21 Planar Graph

```
int n, m;
vector<pll> p;
vi edges[2005];
int ord[2005][2005];
bool flag[2005][2005];

pll origin;
bool cmp_angle(int a, int b) {
    pll p1 = p[a-1], p2 = p[b-1];
    bool flag1 = p1 > origin, flag2 = p2 > origin;
    if(flag1 != flag2) return flag1 > flag2;
    ll curccw = ccw(origin, p1, p2);
    if(curccw != 0) return curccw > 0;
    return false;
}

vector<vector<pll>> find_faces() {
    memset(flag, false, sizeof(flag));

    for(int i=1; i<=n; i++) {
        origin = p[i-1];
        sort(all(edges[i]), cmp_angle);
        for(int j=0; j<edges[i].size(); j++) ord[i][edges[i][j]] = j;
    }

    vector<vector<pll>> faces;
    for(int i=1; i<=n; i++) for(int j=0; j<edges[i].size(); j++)
    if(!flag[i][edges[i][j]]) {
        flag[i][edges[i][j]] = true;

        pll cur = {i, edges[i][j]};
        vector<pll> v; v.push_back(p[i-1]);
        while(true) {
            int sz = (int)edges[cur.second].size();
            int nx_ord = (ord[cur.second][cur.first] - 1 + sz) % sz;
            int nx = edges[cur.second][nx_ord];
            if(flag[cur.second][nx]) break;

            v.push_back(p[cur.second - 1]);
            flag[cur.second][nx] = true;
            cur = {cur.second, nx};
        }
        faces.push_back(v);
    }
    return faces;
}
```

4.22 Shamos-Hoey

```
// a segment endpoint plus the bookkeeping the sweep needs, NOT just a plain point
struct sweep_event {
    pll p; bool isend; int idx;
    sweep_event() { }
    sweep_event(pll _p, bool _isend, int _idx) : p(_p), isend(_isend), idx(_idx) { }
    friend bool operator < (const sweep_event &e1, const sweep_event &e2) {
        return make_pair(make_pair(e1.p.first, e1.isend), make_pair(e1.p.second,
        e1.idx)) < make_pair(make_pair(e2.p.first, e2.isend), make_pair(e2.p.second,
        e2.idx));
    }
};

ll curx;
struct line {
    pll p1, p2;
    pll slope; ll c;
    int idx = -1;
    line() { }
    line(pll _p1, pll _p2) {
        if(_p1 > _p2) swap(_p1, _p2);
        p1 = _p1; p2 = _p2;
        slope = make_pair(p2.first - p1.first, p2.second - p1.second);
        c = slope.first * p1.second - slope.second * p1.first;
    }
    friend bool operator < (const line &l1, const line &l2) {
        bool vert1 = l1.p1.first == l1.p2.first, vert2 = l2.p1.first == l2.p2.first;
        ld y1 = vert1 ? l1.p1.second : (ld) (l1.slope.second * curx + l1.c) /
        l1.slope.first;
        ld y2 = vert2 ? l2.p1.second : (ld) (l2.slope.second * curx + l2.c) /
        l2.slope.first;
        if(y1 != y2) return y1 < y2;

        // tie at the sweep x: break by slope so order matches segments'
        // position just past curx
        if(vert1 != vert2) return vert2; // vertical = steepest
        if(!vert1 && !vert2) {
            ll cross = l1.slope.second * l2.slope.first - l2.slope.second *
            l1.slope.first;
            if(cross != 0) return cross < 0;
        }
        return l1.idx < l2.idx;
    }
};

bool on_segment(pll p, pll a, pll b) {
    if(a > b) swap(a, b);
    return a <= p && p <= b;
}

// count_touching=true: any shared point counts (endpoint touching another
// segment, shared endpoints, collinear overlap)
// count_touching=false: only a genuine transversal crossing through both
// segments' interiors counts
bool isintersect(const line &l1, const line &l2, bool count_touching = true) {
    pll p1 = l1.p1, p2 = l1.p2, p3 = l2.p1, p4 = l2.p2;
    // a degenerate (zero-length) segment can only touch, never properly cross
    if(p1 == p2 || p3 == p4) {
        if(!count_touching) return false;
        if(p1 == p2 && p3 == p4) return p1 == p3;
        if(p1 == p2) return ccw(p3, p4, p1) == 0 && on_segment(p1, p3, p4);
        return ccw(p1, p2, p3) == 0 && on_segment(p3, p1, p2);
    }
    ll ccw1 = ccw(p1, p2, p3), ccw2 = ccw(p1, p2, p4), ccw3 = ccw(p3, p4, p1), ccw4 =
    ccw(p3, p4, p2);
    if(ccw1 == 0 && ccw2 == 0) { // collinear
        if(!count_touching) return false;
        if(p2 < p3 || p4 < p1) return false;
        return true;
    }
    if((ccw1 > 0 && ccw2 > 0) || (ccw1 < 0 && ccw2 < 0)) return false;
    if((ccw3 > 0 && ccw4 > 0) || (ccw3 < 0 && ccw4 < 0)) return false;
    if(!count_touching && (ccw1 == 0 || ccw2 == 0 || ccw3 == 0 || ccw4 == 0)) return
    false;
    return true;
}
int n;
vector<line> lines;
vector<sweep_event> events;
multiset<line> curlines;
unordered_map<int, multiset<line>::iterator> where;

bool shamos_hoey(bool count_touching, pii &ans) {
    n = lines.size();
    events.clear(); curlines.clear(); where.clear();
    for(int i=0; i<n; i++) {
        lines[i].idx = i;
        events.push_back(sweep_event(lines[i].p1, false, i));
        events.push_back(sweep_event(lines[i].p2, true, i));
    }
    sort(all(events));

    for(int i=0; i<n*2; i++) {
        curx = events[i].p.first;
        int idx = events[i].idx;
        if(!events[i].isend) {
            auto it = where[idx] = curlines.insert(lines[idx]);
            if(it != curlines.begin() && isintersect(*prev(it), *it, count_touching))
            {
                ans = {prev(it)->idx, it->idx};
                return true;
            }
            if(next(it) != curlines.end() && isintersect(*it, *next(it),
            count_touching)) {
                ans = {it->idx, next(it)->idx};
                return true;
            }
        }
        else {
            auto it = where[idx];
            bool has_prev = it != curlines.begin(), has_next = next(it) !=
            curlines.end();
            auto pr = has_prev ? prev(it) : it, nx = next(it);
            curlines.erase(it);
            if(has_prev && has_next && isintersect(*pr, *nx, count_touching)) {
                ans = {pr->idx, nx->idx};
                return true;
            }
        }
    }
    return false;
}
```

4.23 Voronoi Diagram

```
/*
Time: O(n log n)
input: order will be changed, sorted by (y,x) order
vertex: voronoi intersection points, degree 3, may duplicated
edge: may contain inf line (-1)
area
  - (a,b) = i-th element of area
  - (u,v) = i-th element of edge
  - input[a] is located CCW of u->v line
  - input[b] is located CW of u->v line
  - u->v line is a subset of perpendicular bisector of input[a] to input[b] segment
  - Straight line {a, b}, {-1, -1} through midpoint of input[a] and input[b]
*/

#define x first
#define y second

ld operator / (pld a, pld b){ return a.first * b.second - a.second * b.first; }
pld operator * (ld b, pld a){ return pld(b * a.first, b * a.second); }
pld operator + (pld a, pld b){ return pld(a.first + b.first, a.second + b.second); }
pld operator - (pld a, pld b){ return pld(a.first - b.first, a.second - b.second); }

const ld EPS = 1e-9;
int dcmp(ld x) { return x < -EPS ? -1 : x > EPS ? 1 : 0; }
// sq(x) = x*x, size(p) = hypot(p.x, p.y)
// sz2(p) = sq(p.x)+sq(p.y), r90(p) = (-p.y, p.x)
ld sq(ld x) { return x * x; }
ld size(pld p) { return hypot(p.x, p.y); }
ld sz2(pld p) { return sq(p.x) + sq(p.y); }
pld r90(pld p) { return pld(-p.y, p.x); }
pld line_intersect(pld a, pld b, pld u, pld v) { return u + (((a - u) / b) / (v / b))
* v; }
pld get_circumcenter(pld p0, pld p1, pld p2) {
    return line_intersect(0.5 * (p0 + p1), r90(p0 - p1), 0.5 * (p1 + p2), r90(p1 -
    p2));
}
ld pb_int(pld left, pld right, ld sweepline) {
    if (dcmp(left.y - right.y) == 0)
        return (left.x + right.x) / 2.0;
    int sign = left.y < right.y ? -1 : 1;
    pld v = line_intersect(left, right - left, pld(0, sweepline), pld(1, 0));
```

```
    ld d1 = sz2(0.5 * (left + right) - v), d2 = sz2(0.5 * (left - right));
    return v.x + sign * sqrt(std::max((ld) 0.0, d1 - d2));
}

struct bnode {
    pld point;
    int idx, end;
    int link[2], par, prv, nxt; // 0 = null
};

bnode nodes[400005]; int node_cnt = 0; // size = 4 * (number of points)

int new_node(pld point, int idx) {
    int cur = ++node_cnt;
    nodes[cur] = {point, idx, 0, {0, 0}, 0, 0, 0};
    return cur;
}

struct Beachline {
    int root = 0;
    ld sweepline = -1e20;

    inline int dir(int n) { return nodes[nodes[n].par].link[0] != n; }
    void rotate(int n) {
        int p = nodes[n].par, d = dir(n);
        nodes[p].link[d] = nodes[n].link[!d];
        if (nodes[n].link[!d]) nodes[nodes[n].link[!d]].par = p;
        nodes[n].par = nodes[p].par;
        if (nodes[p].par) nodes[nodes[p].par].link[dir(p)] = n;
        nodes[n].link[!d] = p;
        nodes[p].par = n;
    }
    void splay(int x, int f = 0) {
        while (nodes[x].par != f) {
            if (nodes[nodes[x].par].par == f);
            else if (dir(x) == dir(nodes[x].par)) rotate(nodes[x].par);
            else rotate(x);
            rotate(x);
        }
        if (!f) root = x;
    }
    void insert(int n, int p, int d) {
        splay(p);
        int c = nodes[p].link[d];
        nodes[n].link[d] = c;
        if (c) nodes[c].par = n;
        nodes[p].link[d] = n;
        nodes[n].par = p;
        int prv = !d ? nodes[p].prv : p, nxt = !d ? p : nodes[p].nxt;
        nodes[n].prv = prv;
        if (prv) nodes[prv].nxt = n;
        nodes[n].nxt = nxt;
        if (nxt) nodes[nxt].prv = n;
    }
    void erase(int n) {
        int prv = nodes[n].prv, nxt = nodes[n].nxt;
        if (!prv && !nxt) {
            if (n == root) root = 0;
            return;
        }
        nodes[n].prv = 0;
        if (prv) nodes[prv].nxt = nxt;
        nodes[n].nxt = 0;
        if (nxt) nodes[nxt].prv = prv;
        splay(n);
        if (!nxt) {
            nodes[root].par = 0;
            nodes[n].link[0] = 0;
            root = prv;
        }
        else {
            splay(nxt, n);
            int c = nodes[n].link[0];
            nodes[nxt].link[0] = c;
            nodes[c].par = nxt;
            nodes[n].link[0] = 0;
            nodes[n].link[1] = 0;
            nodes[nxt].par = 0;
            root = nxt;
        }
    }
    bool get_event(int cur, ld &next_sweep) {
        if (!nodes[cur].prv || !nodes[cur].nxt) return false;
        pld u = r90(nodes[cur].point - nodes[nodes[cur].prv].point);
        pld v = r90(nodes[nodes[cur].nxt].point - nodes[cur].point);
        if (dcmp(u / v) != 1) return false;
        pld p = get_circumcenter(nodes[cur].point, nodes[nodes[cur].prv].point,
        nodes[nodes[cur].nxt].point);
        next_sweep = p.y + size(p - nodes[cur].point);
        return true;
    }
    int find_bl(ld x) {
        int cur = root;
        while (cur) {
            ld left = nodes[cur].prv ? pb_int(nodes[nodes[cur].prv].point,
            nodes[cur].point, sweepline) : -1e30;
            ld right = nodes[cur].nxt ? pb_int(nodes[cur].point,
            nodes[nodes[cur].nxt].point, sweepline) : 1e30;
            if (left <= x && x <= right) {
                splay(cur);
                return cur;
            }
            cur = nodes[cur].link[x > right];
        }
        return 0;
    }
};

struct event {
    int type, idx, prv, nxt, cur;
    ld sweep;
    event(ld sweep, int idx) : type(0), idx(idx), prv(0), nxt(0), cur(0),
    sweep(sweep) { }
    event(ld sweep, int cur, bool circle) : type(1), idx(0),
    prv(nodes[nodes[cur].prv].idx), nxt(nodes[nodes[cur].nxt].idx), cur(cur),
    sweep(sweep) { }
    bool operator>(const event &l) const { return sweep > l.sweep; }
};

void VoronoiDiagram(vector<pld> &input, vector<pld> &vertex, vector<pii> &edge,
vector<pii> &area) {
    Beachline bl = Beachline();
    priority_queue<event, vector<event>, greater<event>> events;
    auto add_edge = [&](int u, int v, int a, int b, int c1, int c2) {
        if (c1) nodes[c1].end = edge.size() * 2;
        if (c2) nodes[c2].end = edge.size() * 2 + 1;
        edge.emplace_back(u, v);
        area.emplace_back(a, b);
    };
    auto write_edge = [&](int idx, int v){
        idx % 2 == 0 ? edge[idx / 2].x = v : edge[idx / 2].y = v;
    };
    auto add_event = [&](int cur) {
        ld nxt;
        if(bl.get_event(cur, nxt)) events.emplace(nxt, cur, true);
    };
    int n = input.size(), cnt = 0;
    node_cnt = 0;
    sort(all(input), [] (const pld &l, const pld &r) {
        return l.y != r.y ? l.y < r.y : l.x < r.x;
    });
    int tmp = bl.root = new_node(input[0], 0), t2;
    for (int i = 1; i < n; i++) {
        if (dcmp(input[i].y - input[0].y) == 0) {
            add_edge(-1, -1, i - 1, i, 0, tmp);
            bl.insert(t2 = new_node(input[i], i), tmp, 1);
            tmp = t2;
        }
        else events.emplace(input[i].y, i);
    }
    while (events.size()) {
        event q = events.top();
        events.pop();
        int prv, cur, nxt, site;
        int v = vertex.size(), idx = q.idx;
        bl.sweepline = q.sweep;
        if (q.type == 0) {
            pld point = input[idx];
            cur = bl.find_bl(point.x);
            bl.insert(site = new_node(point, idx), cur, 0);
            bl.insert(prv = new_node(nodes[cur].point, nodes[cur].idx), site, 0);
            add_edge(-1, -1, nodes[cur].idx, idx, site, prv);
            add_event(prv);
            add_event(cur);
        }
        else {
            cur = q.cur, prv = nodes[cur].prv, nxt = nodes[cur].nxt;
            if (!prv || !nxt || nodes[prv].idx != q.prv || nodes[nxt].idx != q.nxt)
            continue;
            vertex.push_back(get_circumcenter(nodes[prv].point, nodes[nxt].point,
            nodes[cur].point));
            write_edge(nodes[prv].end, v);
            write_edge(nodes[cur].end, v);
            add_edge(v, -1, nodes[prv].idx, nodes[nxt].idx, 0, prv);
            bl.erase(cur);
            add_event(prv);
            add_event(nxt);
        }
    }
}
```

5 Graph

5.1 SPFA

```
// Time: O(VE) worst case, usually close to O(E) (shortest path with negative edges)
int n;
vector<pll> edges[100005];
ll dist[100005]; int cnt[100005]; bool inqueue[100005];

// usage: spfa(s) fills dist[], returns false if a negative cycle is reachable from s
bool spfa(int s) {
    for(int i=1; i<=n; i++) { dist[i] = 4e18; cnt[i] = 0; inqueue[i] = false; }
    queue<int> q; q.push(s);
    dist[s] = 0; inqueue[s] = true;
    while(!q.empty()) {
        int cur = q.front(); q.pop();
        inqueue[cur] = false;
        for(auto [nxt, w] : edges[cur]) {
            if(dist[cur] + w < dist[nxt]) {
                dist[nxt] = dist[cur] + w;
                cnt[nxt] = cnt[cur] + 1;
                if(cnt[nxt] >= n) return false;
                if(!inqueue[nxt]) { inqueue[nxt] = true; q.push(nxt); }
            }
        }
    }
    return true;
}
```

5.2 SCC + 2-SAT

```
vi edges[100005];
int nodenum[100005], ord[100005], cnt = 0;
stack<int> st;
bool visited[100005] = {false};
int groupnum[100005], groupcnt = 0;

void scc(int index) {
    nodenum[index] = ord[index] = ++cnt;
    st.push(index);
    for(int i : edges[index]) {
        if(nodenum[i] == -1) {
            scc(i);
            ord[index] = min(ord[index], ord[i]);
        }
        else if(!visited[i]) ord[index] = min(ord[index], nodenum[i]);
    }
    if(ord[index] == nodenum[index]) {
```

```
        ++groupcnt;
        while(!st.empty()) {
            int cur = st.top(); st.pop();
            groupnum[cur] = groupcnt; visited[cur] = true;
            if(cur == index) break;
        }
    }
}

void SolveProblem() {
    for(int i=1; i<=n*2; i++) nodenum[i] = -1;
    for(int i=1; i<=n*2; i++) if(nodenum[i] == -1) scc(i);

    for(int i=1; i<=n; i++) if(groupnum[i*2-1] == groupnum[i*2]) {
        cout << "-1\n";
        return;
    }

    vector<pii> v;
    for(int i=1; i<=n*2; i++) v.push_back(make_pair(groupnum[i], i));
    sort(all(v));
    for(int i=0; i<n*2; i++) if(!result[(v[i].second+1)/2]) result[(v[i].second+1)/2]
    = v[i].second%2 == 1 ? 'R' : 'B';
    for(int i=1; i<=n; i++) cout << result[i];
    cout << "\n";
}
```

5.3 BCC

```
// (neighbor, edge id); both directions of an edge share the same id
vector<pii> edges[100005];
int nodenum[100005], ord[100005], cnt = 0;
stack<int> st;

int edge_group[100005], groupcnt = 0;

int bcc(int index, int parent_eid) {
    nodenum[index] = ord[index] = ++cnt;
    for(auto [i, eid] : edges[index]) if(eid != parent_eid) {
        if(nodenum[index] > nodenum[i]) st.push(eid);

        if(nodenum[i] != -1) ord[index] = min(ord[index], nodenum[i]);
        else {
            ord[index] = min(ord[index], bcc(i, eid));
            if(ord[i] >= nodenum[index]) {
                groupcnt++;
                while(!st.empty()) {
                    int cur = st.top(); st.pop();
                    edge_group[cur] = groupcnt;
                    if(cur == eid) break;
                }
            }
        }
    }
    return ord[index];
}

void SolveProblem() {
    for(int i=1; i<=n; i++) nodenum[i] = -1;
    for(int i=1; i<=n; i++) if(nodenum[i] == -1) bcc(i, -1);
}
```

5.4 Cactus

```
// Time: O(E log E) (the log factor is from sorting each group)
// (neighbor, edge id); both directions of an edge share the same id
vector<pii> edges[100005];
int nodenum[100005], ord[100005], cnt = 0;
bool visited[100005];
vector<pii> st; // (origin vertex, edge id)
vector<int> group[100005];
int groupnum[100005], groupidx = 0;

void dfs(int index, int parent_eid) {
    nodenum[index] = ++cnt; ord[index] = cnt;
    for(auto [i, eid] : edges[index]) if(eid != parent_eid) {
        if(nodenum[i] == -1) {
            st.push_back({index, eid});
            dfs(i, eid);
            visited[i] = true;
            ord[index] = min(ord[index], ord[i]);
        }
        else if(!visited[i]) {
            ord[index] = min(ord[index], nodenum[i]);
            ++groupidx;

            st.push_back({index, eid});
            while(!st.empty()) {
                pii cur = st.back(); st.pop_back();
                group[groupidx].push_back(cur.second);
                groupnum[cur.second] = groupidx;

                if(cur.first == i) break;
            }
            sort(all(group[groupidx]));
        }
    }
    if(ord[index] == nodenum[index] && !st.empty()) {
        ++groupidx;
        pii cur = st.back(); st.pop_back();
        group[groupidx].push_back(cur.second);
        groupnum[cur.second] = groupidx;
    }
}

// group[g] lists edge ids forming cycle/bridge group g,
// groupnum[edge id] = its group
void SolveProblem() {
    for(int i=1; i<=n; i++) nodenum[i] = -1;
    for(int i=1; i<=n; i++) if(nodenum[i] == -1) dfs(i, -1);
}
```

5.5 Euler Path/Circuit

```
// euler_path: there should be 0 or 2 odd degree nodes
// euler circuit: there should be 0 odd degree nodes
vi edges[100005];
int edge_idx[100005];

void euler(int index) {
    while(edge_idx[index] < edges[index].size()) {
        int cur = edges[index][edge_idx[index]++];
        euler(cur);
    }
    cout << index << " ";
}
```

5.6 Articulation Point

```
int nodenum[100005], ord[100005], cnt = 0;
bool iscut[100005] = {false};

void dfs(int index, int p) {
    nodenum[index] = ord[index] = ++cnt;

    int children = 0;
    for(int i : edges[index]) if(i != p) {
        if(nodenum[i] == -1) {
            children++;
            dfs(i, index);
            if(p != -1 && ord[i] >= nodenum[index]) iscut[index] = true;
            ord[index] = min(ord[index], ord[i]);
        }
        else ord[index] = min(ord[index], nodenum[i]);
    }

    if(p == -1) iscut[index] = children >= 2;
}

void SolveProblem() {
    for(int i=1; i<=n; i++) nodenum[i] = -1;
    for(int i=1; i<=n; i++) if(nodenum[i] == -1) dfs(i, -1);
}
```

5.7 Articulation Bridge

```
int nodenum[100005], ord[100005], cnt = 0;
vector<pii> result;
// usage: edges[u] must hold (v, edge id) pairs so a parallel edge isn't
// mistaken for the parent edge
void dfs(int index, int parent_eid) {
    nodenum[index] = ord[index] = ++cnt;
    for(auto [i, eid] : edges[index]) if(eid != parent_eid) {
        if(nodenum[i] == -1) {
            dfs(i, eid);
            if(ord[i] > nodenum[index]) result.push_back({index, i});
            ord[index] = min(ord[index], ord[i]);
        }
        else ord[index] = min(ord[index], nodenum[i]);
    }
}

void SolveProblem() {
    for(int i=1; i<=n; i++) nodenum[i] = -1;
    for(int i=1; i<=n; i++) if(nodenum[i] == -1) dfs(i, -1);
}
```

5.8 Dominator Tree

```
// Time: iterative dominance computation, O(V+E) per pass; converges in a
// small constant number of passes for typical CFGs (handles cycles, unlike
// a plain topological sweep)
vector<pll> edges[200005], rev_edges[200005];
vi edges_dominator[200005], order; // order = nodes reachable from s, in DFS
postorder
// rpo = reverse-postorder number (smaller = closer to s)
int rpo[200005], idom[200005];
bool visited[200005];

void dfs_order(int index) {
    visited[index] = true;
    for(pll e : edges[index]) if(!visited[e.first]) dfs_order(e.first);
    order.push_back(index);
}

// walk two idom-chain candidates up to their common ancestor, using rpo
// to decide which side climbs
int intersect(int a, int b) {
    while(a != b) {
        while(rpo[a] > rpo[b]) a = idom[a];
        while(rpo[b] > rpo[a]) b = idom[b];
    }
    return a;
}

void dominator_tree(int s) {
    order.clear();
    dfs_order(s);
    reverse(all(order));
    for(int i=0; i<(int) order.size(); i++) rpo[order[i]] = i;
    for(int v : order) idom[v] = -1;
    idom[s] = s;

    bool changed = true;
    while(changed) {
        changed = false;
        for(int v : order) if(v != s) {
            int new_idom = -1;
            for(pll e : rev_edges[v]) {
                if(idom[e.first] == -1) continue;
                new_idom = (new_idom == -1) ? e.first : intersect(new_idom, e.first);
            }
            if(new_idom != -1 && idom[v] != new_idom) { idom[v] = new_idom; changed =
            true; }
        }
    }
    for(int v : order) if(v != s && idom[v] != -1)
    edges_dominator[idom[v]].push_back(v);
}
```

5.9 Offline Dynamic Connectivity

```
int n, m = 0, q;
pii edges[100005]; int in[100005] = {0}, out[100005] = {0};
pii queries[100005]; int query_cnt = 0;

int result[100005] = {0};

vi nodes[400005];
void update(int index, int l, int r, int s, int e, int edge_idx) {
    if(r < s || l > e) return;
    if(s <= l && r <= e) {
        nodes[index].push_back(edge_idx);
        return;
    }
    int mid = (l+r)/2;
    update(index*2, l, mid, s, e, edge_idx); update(index*2+1, mid+1, r, s, e,
    edge_idx);
}

int group[100005];
int find_group(int index) {
    return group[index] == index ? index : find_group(group[index]);
}

stack<int> st;
int sz[100005] = {0};
void Union(int a, int b) {
    if(sz[a] < sz[b]) swap(a, b);
    group[b] = a; sz[a] += sz[b];
    st.push(b);
}

void go_back(int x) {
    while(x--) {
        int a = st.top(); st.pop();
        sz[group[a]] -= sz[a]; group[a] = a;
    }
}

void divide(int index, int l, int r) {
    int cnt = 0;
    for(int edge_idx : nodes[index]) {
        int g1 = find_group(edges[edge_idx].first), g2 =
        find_group(edges[edge_idx].second);
        if(g1 != g2) {
            Union(g1, g2);
            cnt++;
        }
    }

    if(l == r) {
        int g1 = find_group(queries[l].first), g2 = find_group(queries[l].second);
        result[l] = g1 == g2;
        go_back(cnt);
        return;
    }

    int mid = (l+r)/2;
    divide(index*2, l, mid); divide(index*2+1, mid+1, r);
    go_back(cnt);
}

void SolveProblem() {
    cin >> n >> q;

    map<pii, int> idx;
    for(int i=1; i<=q; i++) {
        int type, a, b; cin >> type >> a >> b;
        if(a > b) swap(a, b);
        if(type == 1) {
            edges[++m] = {a, b};
            in[m] = query_cnt+1; out[m] = -1;
            idx[{a, b}] = m;
        }
        else if(type == 2) {
            int cur_idx = idx[{a, b}];
            out[cur_idx] = query_cnt;
        }
        else queries[++query_cnt] = {a, b};
    }
    for(int i=1; i<=m; i++) if(out[i] == -1) out[i] = query_cnt;

    for(int i=1; i<=m; i++) if(in[i] <= out[i]) update(1, 1, query_cnt, in[i],
    out[i], i);

    for(int i=1; i<=n; i++) {
        group[i] = i;
        sz[i] = 1;
    }
    divide(1, 1, query_cnt);

    for(int i=1; i<=query_cnt; i++) cout << result[i] << "\n";
}
```

5.10 Online Dynamic MST

```
void SolveProblem() {
    cin >> n >> q;

    S.clear();
    S.init(n*2+q);
    for(int i=1; i<=n; i++) {
        edges[i].clear();
        S.update_node(i, -1e18);
    }

    result = 0;
    for(int i=2; i<=n; i++) {
        int p; ll d; cin >> p >> d; p++;
        edges[p].push_back({i, d});
        result += d;
    }

    m = n;

    dfs(1, -1);

    while(q--) {
        int a, b; ll d; cin >> a >> b >> d; a++; b++;

        pll mx = S.query(a, b);
        if(mx.first > d) {
            int cur = mx.second;
            int x, y; tie(x, y) = connected[cur];

            result += d - mx.first;
            S.cut(mx.second, connected[cur].first);
            S.cut(mx.second, connected[cur].second);

            ++m;
            S.update_node(m, d);
            connected[m] = {a, b};

            S.reroot(m); S.reroot(a);
            S.link(m, a);

            S.reroot(m); S.reroot(b);
            S.link(m, b);
        }
    }

    cout << result << "\n";
}
```

5.11 kth shortest path (Eppstein's algorithm)

```
// Time: O((V+E) log V + K log K) (Eppstein's algorithm)
struct node { // persistent leftist heap
    ll key; int rank, value;
    int l, r; // 0 = null
};

// size ~ E * log2(E); shrink it to fit the memory limit
node heap[2000005]; int heap_cnt = 0;

int new_node(int rank, ll key, int value, int l, int r) {
    int cur = ++heap_cnt;
    heap[cur] = {key, rank, value, l, r};
    return cur;
}

int insert(int cur, ll key, int value) {
    if(!cur || key < heap[cur].key) return new_node(1, key, value, cur, 0);
    int l = heap[cur].l, r = insert(heap[cur].r, key, value);
    if(!l || heap[r].rank > heap[l].rank) swap(l, r);
    return new_node(r ? heap[r].rank+1 : 1, heap[cur].key, heap[cur].value, l, r);
}

int n;
vector<pii> edges[250005], revedges[250005];
vi edges1[250005];

ll dist[250005]; int parent[250005];
int nodes[250005];
void dijkstra(int e) {
    for(int i=1; i<=n; i++) { dist[i] = 1e18; parent[i] = -1; }
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>> > pq;
    dist[e] = 0; pq.push({dist[e], e});
    while(!pq.empty()) {
        pair<ll, int> tmp = pq.top(); pq.pop();
        if(tmp.first > dist[tmp.second]) continue;

        int cur = tmp.second;
        for(pii e : revedges[cur]) if(dist[cur] + e.second < dist[e.first]) {
            dist[e.first] = dist[cur] + e.second;
            parent[e.first] = cur;
            pq.push({dist[e.first], e.first});
        }
    }
}

ll result[250005];
void find_k_shortest_walk(int s, int e, int k) {
    for(int i=1; i<=k; i++) result[i] = -1;

    for(int i=1; i<=n; i++) for(pii e : edges[i]) revedges[e.first].push_back({i,
    e.second});
    dijkstra(e);

    if(dist[s] == 1e18) return;

    for(int i=1; i<=n; i++) if(parent[i] != -1) edges1[parent[i]].push_back(i);

    queue<int> q; q.push(e);
    while(!q.empty()) {
        int cur = q.front(); q.pop();
        bool flag = false;
        for(pii e : edges[cur]) {
            if(dist[e.first] == 1e18) continue;
            ll cost = e.second + dist[e.first] - dist[cur];
            if(!flag && e.first == parent[cur] && cost == 0) {
                flag = true;
                continue;
            }
            nodes[cur] = insert(nodes[cur], cost, e.first);
        }
        for(int nx : edges1[cur]) {
            nodes[nx] = nodes[cur];
            q.push(nx);
        }
    }

    result[1] = dist[s];

    if(!nodes[s]) return;

    priority_queue<pll, vector<pll>, greater<pll> > pq;
    pq.push({dist[s] + heap[nodes[s]].key, nodes[s]});

    for(int i=2; i<=k && !pq.empty(); i++) {
```

```
        pll cur = pq.top(); pq.pop();
        result[i] = cur.first;

        int curnode = cur.second;
        if(nodes[heap[curnode].value]) pq.push({cur.first +
        heap[nodes[heap[curnode].value]].key, nodes[heap[curnode].value]});
        if(heap[curnode].l) pq.push({cur.first + heap[heap[curnode].l].key -
        heap[curnode].key, heap[curnode].l});
        if(heap[curnode].r) pq.push({cur.first + heap[heap[curnode].r].key -
        heap[curnode].key, heap[curnode].r});
    }
}

int N, M, s, e;
ll arr[250005];
ll ans = 0;

void SolveProblem() {
    n = N+2;
    find_k_shortest_walk(s, e, M);

    for(int i=1; i<=M; i++) cout << result[i] << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    int test_case = 1;
    // cin >> test_case;
    for(int tc = 1; tc <= test_case; tc++) {
        SolveProblem();
    }
}
```

6 Tree

6.1 Heavy Light Decomposition (HLD)

```
int parent[500005], depth[500005], sz[500005];
int chain_num[500005], chain_idx[500005];
int in[500005], out[500005]; int cnt = 0;
vi chain[500005];

void dfs(int index, int p, int d) {
    parent[index] = p; depth[index] = d;
    sz[index] = 1;
    for(int i : edges[index]) if(i != p) {
        dfs(i, index, d+1);
        sz[index] += sz[i];
    }
}

void hld(int index, int p, int cur_chain) {
    chain_num[index] = cur_chain;
    chain_idx[index] = chain[cur_chain].size();
    chain[cur_chain].push_back(index);

    in[index] = ++cnt;

    int heavy = -1;
    for(int i : edges[index]) if(i != p && (heavy == -1 || sz[i] > sz[heavy])) heavy
    = i;

    if(heavy != -1) hld(heavy, index, cur_chain);
    for(int i : edges[index]) if(i != p && i != heavy) hld(i, index, i);

    out[index] = cnt;
}

void SolveProblem() {
    dfs(1, -1, 0);
    hld(1, -1, 1);
}
```

6.2 Centroid Decomposition

```
void find_sz(int index, int p) {
    sz[index] = 1;
    for(int i : edges[index]) if(i != p && !visited[i]) {
        find_sz(i, index);
        sz[index] += sz[i];
    }
}

int find_centroid(int index, int p, int cursz) {
    for(int i : edges[index]) if(i != p && !visited[i]) {
        if(sz[i]*2 > cursz) return find_centroid(i, index, cursz);
    }
    return index;
}

void dfs(int index, int parent) {
    // ... gather/query info within the centroid's current component
    // (fill in per-problem)
}

void centroid_dnc(int index) {
    find_sz(index, -1);

    int centroid = find_centroid(index, -1, sz[index]);
    for(int i : edges[centroid]) if(!visited[i]) {
        // v.clear();
        dfs(i, centroid);
        // ...
    }

    visited[centroid] = true;
    for(int i : edges[centroid]) if(!visited[i]) centroid_dnc(i);
}
```

6.3 Tree Compression

```
vector<pii> edges1[500005];

int build_virtual_tree(vi &nodes) {
    sort(all(nodes)); nodes.erase(unique(all(nodes)), nodes.end());
    sort(all(nodes), cmp_in);

    int k = nodes.size();
    for(int i=0; i<k-1; i++) nodes.push_back(find_lca(nodes[i], nodes[i+1]));
    sort(all(nodes)); nodes.erase(unique(all(nodes)), nodes.end());
    sort(all(nodes), cmp_in);

    for(int i=0; i<(int)nodes.size()-1; i++) {
        int a = nodes[i], b = nodes[i+1];
        int p = find_lca(a, b);
        edges1[p].push_back({b, depth[b]-depth[p]});
    }
    return nodes[0];
}
```

6.4 Tree Isomorphism

```
const ll MOD1 = 1e9+7, MOD2 = 998244353;
const ll p1 = 19192734, p2 = 603820;
ll exp_p1[200005], exp_p2[200005];

void precompute_pow() {
    exp_p1[0] = exp_p2[0] = 1;
    for(int i=1; i<200005; i++) { exp_p1[i] = exp_p1[i-1] * p1 % MOD1; exp_p2[i] =
    exp_p2[i-1] * p2 % MOD2; }
}

void find_sz(int index, int p) {
    sz[index] = 1;
    for(int i : edges[index]) if(i != p) {
        find_sz(i, index);
        sz[index] += sz[i];
    }
}

void find_centroid(int index, int p) {
    bool valid = true;
    for(int i : edges[index]) if(i != p) {
        if(sz[i] * 2 >= m) find_centroid(i, index);
        if(sz[i] * 2 > m) valid = false;
    }

    if(valid) centroid.push_back(index);
}

void find_hash(int index, int p) {
    // "(" : 28, ")": 29
    hash_tree[index] = {28, 28};

    vector<pair<pll, int> > v;
    for(int i : edges[index]) if(i != p) {
        find_hash(i, index);
        v.push_back({hash_tree[i], i});
    }
    sort(all(v));

    int cur_sz = 1;
    for(int i=0; i<v.size(); i++) {
        pll a = v[i].first; int idx = v[i].second;
        hash_tree[index].first = (hash_tree[index].first + exp_p1[cur_sz] * a.first %
        MOD1) % MOD1;
        hash_tree[index].second = (hash_tree[index].second + exp_p2[cur_sz] *
        a.second % MOD2) % MOD2;
        cur_sz += sz[idx]*2;
    }

    hash_tree[index].first = (hash_tree[index].first + exp_p1[cur_sz] * 29 % MOD1) %
    MOD1;
    hash_tree[index].second = (hash_tree[index].second + exp_p2[cur_sz] * 29 % MOD2)
    % MOD2;
}

void SolveProblem() {
    precompute_pow();
    find_sz(1, -1);

    centroid.clear();
    find_centroid(1, -1);

    if(centroid.size() == 2) {
        int a = centroid[0], b = centroid[1];
        for(int i=0; i<edges[a].size(); i++) if(edges[a][i] == b) {
            edges[a].erase(edges[a].begin() + i);
            break;
        }
        for(int i=0; i<edges[b].size(); i++) if(edges[b][i] == a) {
            edges[b].erase(edges[b].begin() + i);
            break;
        }
        edges[a].push_back(m+1); edges[m+1].push_back(a);
        edges[b].push_back(m+1); edges[m+1].push_back(b);
    }

    int cur = centroid.size() == 1 ? centroid[0] : m+1;
    find_sz(cur, -1);
    find_hash(cur, -1);
}
```

7 Flow / Matching

7.1 Bipartite Matching (Hopcroft-Karp)

```
// Time: O(E sqrt(V))
int connected[200005], rev_connected[200005];
bool visited[200005] = {false}; int dist[200005] = {0};
void bfs() {
    queue<int> q;
    for(int i=1; i<=n; i++) {
        visited[i] = false;
        if(connected[i] == -1) {
            dist[i] = 0;
```

```
            q.push(i);
        }
        else dist[i] = 1e9;
    }

    while(!q.empty()) {
        int cur = q.front(); q.pop();
        for(int i : edges[cur]) {
            if(rev_connected[i] != -1 && dist[rev_connected[i]] == 1e9) {
                dist[rev_connected[i]] = dist[cur] + 1;
                q.push(rev_connected[i]);
            }
        }
    }
}

bool bip_match(int index) {
    if(visited[index]) return false;
    visited[index] = true;

    for(int i : edges[index]) {
        if(rev_connected[i] == -1 || dist[rev_connected[i]] == dist[index] + 1 &&
        bip_match(rev_connected[i])) {
            connected[index] = i;
            rev_connected[i] = index;
            return true;
        }
    }
    return false;
}

bool team_l[200005], team_r[200005];
// usage: after matching, team_l[i] / team_r[i] = true if on the source side of min
cut
void find_team() {
    queue<int> q;
    for(int i=1; i<=n; i++) {
        team_l[i] = connected[i] == -1;
        if(team_l[i]) q.push(i);
    }
    for(int i=1; i<=m; i++) team_r[i] = false;

    while(!q.empty()) {
        int cur = q.front(); q.pop();
        for(int i : edges[cur]) {
            if(team_r[i]) continue;
            team_r[i] = true;
            team_l[rev_connected[i]] = true;
            q.push(rev_connected[i]);
        }
    }
}

void SolveProblem() {
    for(int i=1; i<=n; i++) connected[i] = -1;
    for(int i=1; i<=m; i++) rev_connected[i] = -1;
    while(1) {
        bfs();

        bool flag = false;
        for(int i=1; i<=n; i++) if(connected[i] == -1 && bip_match(i)) flag = true;

        if(!flag) break;
    }
    find_team();
}
```

7.2 Matching / Matroid Lemmas

- **Hall's Marriage Theorem**
  A bipartite graph $G = (X, Y, E)$ has a matching that covers every vertex in X if and only if for every subset $S \subseteq X$, $|N(S)| \geq |S|$, where $N(S)$ is the set of neighbors of S.

- **Kőnig's Theorem**
  In a bipartite graph $G = (X, Y, E)$,

$$|\text{maximum matching}| = |\text{minimum vertex cover}|$$

  By LP duality / Hall's theorem, this also gives $|\text{maximum independent set}| = |X| + |Y| - |\text{maximum matching}|$.

  Constructing a minimum vertex cover from a maximum matching:
  Let M be a maximum matching. From every unmatched vertex in X, run a DFS/BFS along alternating paths (unmatched edges from X to Y, matched edges from Y back to X), and let Z be the set of all vertices reached (including the unmatched starting vertices). Then

$$\text{minimum vertex cover} = (X \setminus Z) \cup (Y \cap Z)$$

  i.e. take the X-side vertices not reached by an alternating path, and the Y-side vertices that are reached. Its complement, $(X \cap Z) \cup (Y \setminus Z)$, is a maximum independent set.

- **Matching is a Linear Matroid**
  In a bipartite graph $G = (X, Y, E)$, the family of subsets $A \subseteq X$ that can be saturated by some matching (the transversal matroid of G) is representable as a linear matroid: assign each $y \in Y$ a random vector v_y (e.g. random values mod a large prime), and each $x \in X$ the vector $u_x = \sum_{(x,y)\in E} v_y$. Then, with high probability, $A \subseteq X$ is matchable $\iff$ $\{u_x : x \in A\}$ is linearly independent, so

$$|\text{maximum matching}| = \operatorname{rank}\left((u_x)_{x\in X}\right)$$

  This is the theoretical basis for randomized algebraic matching: the biadjacency matrix (bipartite case) or Tutte matrix (general graphs, as in `general_matching.cpp`) has rank equal to twice the max matching (general graphs) or exactly the max matching (bipartite), with failure probability $\leq n/p$ for a random prime/modulus p.

7.3 Max Flow (Dinic)

```
// Time: O(V^2 E) general graphs; O(E sqrt(V)) for unit-capacity graphs
// (e.g. bipartite matching)
const int MAXV = 505;
ll capacity[MAXV][MAXV], flow[MAXV][MAXV];
vi edges[MAXV]; int level[MAXV], idx[MAXV], source, sink, teamnum[MAXV]; ll result =
0;

bool find_level() {
    memset(level, -1, sizeof level);
    queue<int> q; q.push(source);
    level[source] = 0;
    while(!q.empty()) {
        int cur = q.front(); q.pop();
        for(int i : edges[cur]) {
            if(level[i] == -1 && capacity[cur][i] - flow[cur][i] > 0) {
                level[i] = level[cur] + 1;
                q.push(i);
            }
        }
    }
    return level[sink] != -1;
}
ll find_flow(int cur, ll curflow) {
    if(cur == sink) return curflow;
    for(int &i = idx[cur]; i<edges[cur].size(); i++) {
        int nx = edges[cur][i];
        if(capacity[cur][nx] - flow[cur][nx] > 0 && level[nx] == level[cur] + 1) {
            ll amount = find_flow(nx, min(curflow, capacity[cur][nx] -
            flow[cur][nx]));
            if(amount > 0) {
                flow[cur][nx] += amount;
                flow[nx][cur] -= amount;
                return amount;
            }
        }
    }
    return 0;
}

void find_team() {
    queue<int> q; q.push(source);
    teamnum[source] = 1;
    while(!q.empty()) {
        int cur = q.front(); q.pop();
        for(int i : edges[cur]) {
            if(!teamnum[i] && capacity[cur][i] - flow[cur][i] > 0) {
                teamnum[i] = 1;
                q.push(i);
            }
        }
    }
}

void SolveProblem() {
    while(find_level()) {
        memset(idx, 0, sizeof idx);
        while(1) {
            ll amount = find_flow(source, 1e9);
            if(amount == 0) break;
            result += amount;
        }
    }
    find_team();
}
```

7.4 Min Cost Max Flow (MCMF) - Dijkstra + Potentials

```
// Time: O(F * E log V), F = max flow value
int n, source, sink;
vi edges[405];
ll capacity[405][405], flow[405][405], cost[405][405];
ll dist[405], pi[405]; int parent[405]; bool visited[405];

void setpi() { // one Bellman-Ford pass; only needed if some cost[][] starts negative
    fill(pi+1, pi+sink+1, (ll) 4e18); pi[source] = 0;
    for(int cnt=1; cnt<=sink; cnt++) {
        bool changed = false;
        for(int i=1; i<=sink; i++) if(pi[i] < (ll) 4e18) for(int j : edges[i])
        if(capacity[i][j] > 0) {
            ll v = pi[i] + cost[i][j];
            if(v < pi[j]) { pi[j] = v; changed = true; }
        }
        if(!changed) break;
    }
}

bool dijkstra() {
    fill(dist+1, dist+sink+1, (ll) 4e18);
    fill(visited+1, visited+sink+1, false);
    priority_queue<pll, vector<pll>, greater<pll>> pq;
    dist[source] = 0; pq.push({0, source});
    while(!pq.empty()) {
        auto [d, cur] = pq.top(); pq.pop();
        if(visited[cur]) continue;
        visited[cur] = true;
        for(int i : edges[cur]) {
            if(capacity[cur][i] - flow[cur][i] <= 0) continue;
```

```
            ll nd = d + cost[cur][i] + pi[cur] - pi[i]; // reduced cost, always >= 0
            if(nd < dist[i]) { dist[i] = nd; parent[i] = cur; pq.push({nd, i}); }
        }
    }
    for(int i=1; i<=sink; i++) if(dist[i] < (ll) 4e18) pi[i] += dist[i];
    return dist[sink] < (ll) 4e18;
}

// usage: source = n+1, sink = n+2 before adding edges; returns {max_flow, min_cost}
pll mcmf() {
    ll max_flow = 0, min_cost = 0;
    while(dijkstra()) {
        ll amount = (ll) 4e18;
        for(int p=sink; p!=source; p=parent[p]) amount = min(amount,
        capacity[parent[p]][p] - flow[parent[p]][p]);
        for(int p=sink; p!=source; p=parent[p]) {
            min_cost += amount * cost[parent[p]][p];
            flow[parent[p]][p] += amount;
            flow[p][parent[p]] -= amount;
        }
        max_flow += amount;
    }
    return {max_flow, min_cost};
}
```

7.5 Stable Marriage Problem

```
// cus_like[i] = customer i's resources in preference order
// (1-indexed, most preferred first)
// res_pref[r][i] = resource r's rank for customer i (lower = better)
// Time: O(n^2)
int n;
vi cus_like[1005];
int res_pref[1005][1005], cap[1005];
priority_queue<pii> res_to_cus[1005];
int idx[1005], cus_to_res[1005];

void gale_shapley() {
    for(int t=1; t<=n; t++) {
        for(int i=1; i<=n; i++) {
            if(cus_to_res[i]) continue;

            while(idx[i] <= n) {
                int res = cus_like[i][idx[i]];
                if(res_to_cus[res].size() < cap[res]) {
                    cus_to_res[i] = res;
                    res_to_cus[res].push({res_pref[res][i], i});
                    break;
                }
                else if(cap[res]) {
                    pii last = res_to_cus[res].top();
                    if(res_pref[res][i] < last.first) {
                        cus_to_res[last.second] = 0;
                        res_to_cus[res].pop();
                        cus_to_res[i] = res;
                        res_to_cus[res].push({res_pref[res][i], i});
                        break;
                    }
                }
                idx[i]++;
            }
        }
    }
}

// Example usage: n customers and n resources (capacities may exceed 1).
// Reads, in order:
//   each customer's preference list over resources (n ids, most-preferred first),
//   each resource's capacity,
//   each resource's preference order over customers (n ids, most-preferred first).
// Prints each customer's matched resource, 0 if unmatched.
void SolveProblem() {
    cin >> n;
    for(int i=1; i<=n; i++) {
        cus_like[i].resize(n+1);
        idx[i] = 1;
        for(int j=1; j<=n; j++) cin >> cus_like[i][j];
    }
    for(int r=1; r<=n; r++) cin >> cap[r];
    for(int r=1; r<=n; r++) for(int k=1; k<=n; k++) {
        int cus; cin >> cus;
        res_pref[r][cus] = k;
    }

    gale_shapley();

    for(int i=1; i<=n; i++) cout << cus_to_res[i] << " ";
    cout << "\n";
}
```

7.6 Circulation (Lower Bound Flow)

```
// Feasible circulation with lower bounds, on top of Dinic. Time: O(V^2 E)
// usage: set n; add_edge(u, v, lower, upper) for every edge;
// bool ok = feasible_circulation();
// if ok, actual flow on each edge = lower + (upper-lower - remaining .cap
// on that edge's Dinic entry)
// Each edge keeps flow in [lower, upper]; every vertex must conserve flow.
// Flow actually pushed on an edge = lower + (upper - lower - e.cap).
struct edge {
    int to, rev;
    ll cap;
};
int n; // real vertices are 1..n, super source/sink are n+1, n+2
vector<edge> edges[405];
int level[405], idx[405];
ll excess[405];

void add_edge(int u, int v, ll lower, ll upper) {
    edges[u].push_back({v, (int) edges[v].size(), upper - lower});
    edges[v].push_back({u, (int) edges[u].size()-1, 0});
    excess[v] += lower; excess[u] -= lower;
}

bool find_level(int source, int sink) {
    memset(level, -1, sizeof level);
    queue<int> q; q.push(source);
    level[source] = 0;
    while(!q.empty()) {
        int cur = q.front(); q.pop();
        for(edge &e : edges[cur]) if(level[e.to] == -1 && e.cap > 0) {
            level[e.to] = level[cur] + 1;
            q.push(e.to);
        }
    }
    return level[sink] != -1;
}

ll find_flow(int cur, int sink, ll curflow) {
    if(cur == sink) return curflow;
    for(int &i = idx[cur]; i < edges[cur].size(); i++) {
        edge &e = edges[cur][i];
        if(e.cap > 0 && level[e.to] == level[cur] + 1) {
            ll amount = find_flow(e.to, sink, min(curflow, e.cap));
            if(amount > 0) {
                e.cap -= amount;
                edges[e.to][e.rev].cap += amount;
                return amount;
            }
        }
    }
    return 0;
}

ll max_flow(int source, int sink) {
    ll result = 0;
    while(find_level(source, sink)) {
        memset(idx, 0, sizeof idx);
        while(ll amount = find_flow(source, sink, 4e18)) result += amount;
    }
    return result;
}

bool feasible_circulation() {
    int ss = n+1, tt = n+2;
    ll need = 0;
    for(int i=1; i<=n; i++) {
        if(excess[i] > 0) { add_edge(ss, i, 0, excess[i]); need += excess[i]; }
        else if(excess[i] < 0) add_edge(i, tt, 0, -excess[i]);
    }
    return max_flow(ss, tt) == need;
}

// Max s-t flow with lower bounds:
//   add_edge(t, s, 0, 4e18) as the last edge, then feasible_circulation();
//   the flow on that edge is a feasible s-t flow, and max_flow(s, t) on the
//   leftover residual graph increases it to the maximum.
// Min s-t flow: use max_flow(t, s) on the residual instead.

// Example usage: max flow from s to t where each edge has a lower and upper bound.
void SolveProblem() {
    int m, s, t; cin >> n >> m >> s >> t;
    for(int i=1; i<=m; i++) {
        int u, v; ll lower, upper; cin >> u >> v >> lower >> upper;
        add_edge(u, v, lower, upper);
    }

    const ll INF = 4e18;
    int back_idx = edges[t].size();
    // any feasible circulation now carries a feasible s-t flow on this edge
    add_edge(t, s, 0, INF);

    if(!feasible_circulation()) {
        cout << "infeasible\n";
        return;
    }

    // push more flow on whatever residual capacity is left; this can shuffle
    // flow off of the back edge, so read it after
    ll extra = max_flow(s, t);
    // total flow now sitting on the back edge
    ll base_flow = INF - edges[t][back_idx].cap;

    cout << base_flow + extra << "\n";
}
```

7.7 Global Min cut (Stoer-Wagner)

```
// Global min cut of an undirected graph, O(V^3). 'adj' is destroyed by the merges.
// usage: set n and adj[1..n][1..n] (symmetric edge weights, 0 for no edge);
// ll cut = stoer_wagner();
int n; ll adj[505][505];
bool merged[505], added[505];
ll cut_weight[505];

ll stoer_wagner() {
    ll result = 4e18;
    for(int i=1; i<=n; i++) merged[i] = false;

    for(int phase=1; phase<n; phase++) {
        for(int i=1; i<=n; i++) {
            cut_weight[i] = 0;
            added[i] = false;
        }

        int prev = -1, last = -1;
        for(int it=1; it<=n-phase+1; it++) {
            int sel = -1;
            for(int i=1; i<=n; i++) if(!merged[i] && !added[i] && (sel == -1 ||
            cut_weight[i] > cut_weight[sel])) sel = i;

            added[sel] = true;
            prev = last; last = sel;
            for(int i=1; i<=n; i++) if(!merged[i] && !added[i]) cut_weight[i] +=
            adj[sel][i];
        }

        result = min(result, cut_weight[last]);
```

```
        // merge 'last' into 'prev'
        merged[last] = true;
        for(int i=1; i<=n; i++) {
            adj[prev][i] += adj[last][i];
            adj[i][prev] = adj[prev][i];
        }
    }

    return result;
}
```

7.8 Gomory-Hu Tree (All-Pairs Min Cut)

```
// Gomory-Hu Tree (Gusfield's simplification): the max flow / min cut
// between ANY pair of vertices in the original undirected graph equals
// the minimum edge weight along their path in this tree.
// Time: O(V) max-flow computations -> O(V^3 E) total with the O(V^2 E) Dinic below.
int n;
vi edges[505];
ll capacity[505][505], flow[505][505];
int level[505], idx[505];
// par[i]/tree_weight[i] = i's parent and edge weight in the GH tree (i=2..n)
int par[505]; ll tree_weight[505];

void add_edge(int u, int v, ll w) {
    if(!capacity[u][v] && !capacity[v][u]) { edges[u].push_back(v);
    edges[v].push_back(u); }
    capacity[u][v] += w; capacity[v][u] += w;
}

bool find_level(int source, int sink) {
    memset(level, -1, sizeof level);
    queue<int> q; q.push(source);
    level[source] = 0;
    while(!q.empty()) {
        int cur = q.front(); q.pop();
        for(int i : edges[cur]) if(level[i] == -1 && capacity[cur][i] - flow[cur][i]
        > 0) {
            level[i] = level[cur] + 1;
            q.push(i);
        }
    }
    return level[sink] != -1;
}
ll find_flow(int cur, int sink, ll curflow) {
    if(cur == sink) return curflow;
    for(int &i = idx[cur]; i < edges[cur].size(); i++) {
        int nx = edges[cur][i];
        if(capacity[cur][nx] - flow[cur][nx] > 0 && level[nx] == level[cur] + 1) {
            ll amount = find_flow(nx, sink, min(curflow, capacity[cur][nx] -
            flow[cur][nx]));
            if(amount > 0) {
                flow[cur][nx] += amount;
                flow[nx][cur] -= amount;
                return amount;
            }
        }
    }
    return 0;
}
// after this returns, level[j] != -1 means j is on source's side of the
// resulting min cut
ll max_flow(int source, int sink) {
    memset(flow, 0, sizeof flow);
    ll result = 0;
    while(find_level(source, sink)) {
        memset(idx, 0, sizeof idx);
        while(ll amount = find_flow(source, sink, (ll) 4e18)) result += amount;
    }
    return result;
}

void build_gomory_hu() {
    for(int i=2; i<=n; i++) par[i] = 1;
    for(int i=2; i<=n; i++) {
        tree_weight[i] = max_flow(i, par[i]);
        for(int j=i+1; j<=n; j++) if(par[j] == par[i] && level[j] != -1) par[j] = i;
    }
}

// max flow / min cut between a and b, read off the tree (min edge weight
// along the a..b path)
ll query(int a, int b) {
    unordered_map<int, ll> seen;
    ll cur = (ll) 4e18;
    int x = a;
    seen[x] = cur;
    while(par[x]) {
        cur = min(cur, tree_weight[x]);
        x = par[x];
        seen[x] = cur;
    }
    cur = (ll) 4e18;
    x = b;
    while(!seen.count(x)) {
        cur = min(cur, tree_weight[x]);
        x = par[x];
    }
    return min(cur, seen[x]);
}

// usage: set n; add_edge(u, v, w) for every undirected edge; build_gomory_hu(); then
// query(a, b) answers the max flow / min cut between a and b in O(depth) each
void SolveProblem() {
    int m, q; cin >> n >> m;
    for(int i=1; i<=m; i++) {
        int u, v; ll w; cin >> u >> v >> w;
        add_edge(u, v, w);
    }

    build_gomory_hu();

    cin >> q;
    for(int i=1; i<=q; i++) {
        int a, b; cin >> a >> b;
        cout << query(a, b) << "\n";
    }
}
```

7.9 General Matching (Blossom)

```
// Maximum matching on a general graph (Blossom algorithm), O(V^3)
// usage: set n and edges[u] (undirected, add both directions);
// int m = general_matching();
// match[i] = i's partner (-1 if unmatched)
int n;
vi edges[505];
int match[505], p[505], base[505];
bool used[505], blossom[505];

void mark_path(int v, int b, int child) {
    while(base[v] != b) {
        blossom[base[v]] = true;
        blossom[base[match[v]]] = true;
        p[v] = child;
        child = match[v];
        v = p[match[v]];
    }
}

int lca(int a, int b) {
    static bool used_lca[505];
    memset(used_lca, false, sizeof used_lca);

    int v = a;
    while(1) {
        v = base[v]; used_lca[v] = true;
        if(match[v] == -1) break;
        v = p[match[v]];
    }

    v = b;
    while(1) {
        v = base[v];
        if(used_lca[v]) return v;
        v = p[match[v]];
    }
}

// returns the endpoint of an augmenting path from 'root', -1 if none
int find_path(int root) {
    memset(used, false, sizeof used);
    memset(p, -1, sizeof p);
    for(int i=1; i<=n; i++) base[i] = i;

    used[root] = true;
    queue<int> q; q.push(root);
    while(!q.empty()) {
        int v = q.front(); q.pop();
        for(int to : edges[v]) {
            if(base[v] == base[to] || match[v] == to) continue;

            if(to == root || (match[to] != -1 && p[match[to]] != -1)) {
                int curbase = lca(v, to);
                memset(blossom, false, sizeof blossom);
                mark_path(v, curbase, to);
                mark_path(to, curbase, v);
                for(int i=1; i<=n; i++) if(blossom[base[i]]) {
                    base[i] = curbase;
                    if(!used[i]) { used[i] = true; q.push(i); }
                }
            }
            else if(p[to] == -1) {
                p[to] = v;
                if(match[to] == -1) return to;
                used[match[to]] = true; q.push(match[to]);
            }
        }
    }
    return -1;
}

int general_matching() {
    for(int i=1; i<=n; i++) match[i] = -1;

    for(int i=1; i<=n; i++) if(match[i] == -1) {
        int v = find_path(i);
        while(v != -1) {
            int pv = p[v], ppv = match[pv];
            match[v] = pv; match[pv] = v;
            v = ppv;
        }
    }

    int result = 0;
    for(int i=1; i<=n; i++) if(match[i] != -1) result++;
    return result / 2;
}
```

7.10 Hungarian Algorithm ($O(n^3)$)

```
int n;
int max_match = 0;
ll cost[505][505];
ll label_x[505], label_y[505];
int match_x[505], match_y[505];
pll slack[505];
bool S[505], T[505];
int parent[505];

void init_label() {
    for(int i=1; i<=n; i++) label_x[i] = label_y[i] = 0;
    for(int i=1; i<=n; i++) for(int j=1; j<=n; j++) label_x[i] = max(label_x[i],
    cost[i][j]);
}

void update_label() {
    ll min_label = 1e18;
    for(int j=1; j<=n; j++) if(!T[j]) min_label = min(min_label, slack[j].first);
```

```
    for(int i=1; i<=n; i++) if(S[i]) label_x[i] -= min_label;
    for(int j=1; j<=n; j++) if(T[j]) label_y[j] += min_label;

    for(int j=1; j<=n; j++) if(!T[j]) slack[j].first -= min_label;
}

void add_to_tree(int cur, int p) {
    S[cur] = true; parent[cur] = p;
    for(int j=1; j<=n; j++) if(label_x[cur] + label_y[j] - cost[cur][j] <
    slack[j].first) {
        slack[j].first = label_x[cur] + label_y[j] - cost[cur][j];
        slack[j].second = cur;
    }
}

void augment() {
    if(max_match == n) return;
    int x, y = n+1, root;
    for(int i=1; i<=n; i++) {
        S[i] = T[i] = false;
        parent[i] = -1;
    }

    queue<int> q;
    for(int i=1; i<=n; i++) if(match_x[i] == -1) {
        root = i; parent[root] = -2; S[root] = true;
        q.push(root);
        break;
    }

    for(int j=1; j<=n; j++) slack[j] = {label_x[root] + label_y[j] - cost[root][j],
    root};

    while(1) {
        while(!q.empty()) {
            x = q.front(); q.pop();

            for(int j=1; j<=n; j++) {
                if(label_x[x] + label_y[j] - cost[x][j] == 0 && !T[j]) {
                    if(match_y[j] == -1) {
                        y = j;
                        break;
                    }
                    T[j] = true;
                    q.push(match_y[j]);
                    add_to_tree(match_y[j], x);
                }
            }

            if(y <= n) break;
        }

        if(y <= n) break;
        update_label();
        while(!q.empty()) q.pop();

        for(int j=1; j<=n; j++) {
            if(slack[j].first == 0 && !T[j]) {
                if(match_y[j] == -1) {
                    x = slack[j].second;
                    y = j;
                    break;
                }
                else {
                    T[j] = true;
                    if(!S[match_y[j]]) {
                        q.push(match_y[j]);
                        add_to_tree(match_y[j], slack[j].second);
                    }
                }
            }
        }

        if(y <= n) break;
    }

    if(y <= n) {
        max_match++;
        for(int cx = x, cy = y, ty; cx != -2; cx = parent[cx], cy = ty) {
            ty = match_x[cx];
            match_x[cx] = cy; match_y[cy] = cx;
        }
        augment();
    }
}

ll hungarian() {
    for(int i=1; i<=n; i++) for(int j=1; j<=n; j++) cost[i][j] = -cost[i][j];

    max_match = 0;
    for(int i=1; i<=n; i++) match_x[i] = match_y[i] = -1;
    init_label();
    augment();

    ll result = 0;
    for(int i=1; i<=n; i++) result += cost[i][match_x[i]];

    return -result;
}
```

8 String

8.1 KMP

```
void calc_lps() {
    int j=2, len = 0;
    lps[1] = 0;
    while(j <= m) {
        if(s2[j] == s2[len+1]) lps[j++] = ++len;
        else {
            if(len) len = lps[len];
            else lps[j++] = 0;
        }
    }
}

void SolveProblem() {
    cin >> s1;
    n = s1.size(); s1 = " " + s1;
    cin >> s2;
    m = s2.size(); s2 = " " + s2;

    calc_lps();

    for(int i=1; i<=n; i++) {
        while(1) {
            if(j+1 <= m && s1[i] == s2[j+1]) {
                j++;
                break;
            }
            else {
                if(j) j = lps[j];
                else break;
            }
        }
    }
}
```

8.2 Aho-Corasick

```
struct Trie {
    map<int, int> child;
    int fail;
    bool isend; int isend_cnt;
    Trie() {
        fail = 0;
        isend = false; isend_cnt = 0;
    }
};

Trie nodes[305]; int node_cnt = 0;
int root;

void insert(string &s, int cur, int index) {
    if(s.size() == index) {
        nodes[cur].isend = true;
        nodes[cur].isend_cnt++;
        return;
    }

    int x = s[index]-'A'+1; // 1-indexed: 'A'..'Z' -> 1..26
    if(!nodes[cur].child.count(x)) nodes[cur].child[x] = ++node_cnt;
    insert(s, nodes[cur].child[x], index+1);
}

void Aho_Corasick() {
    queue<int> q; q.push(root);
    nodes[root].fail = root;

    while(!q.empty()) {
        int cur = q.front(); q.pop();
        for(auto [i, next] : nodes[cur].child) {
            if(cur == root) nodes[next].fail = root;
            else {
                int dest = nodes[cur].fail;
                while(dest != root && !nodes[dest].child.count(i)) dest =
                nodes[dest].fail;
                if(nodes[dest].child.count(i)) dest = nodes[dest].child[i];
                nodes[next].fail = dest;
            }

            if(nodes[nodes[next].fail].isend) {
                nodes[next].isend = true;
                nodes[next].isend_cnt += nodes[nodes[next].fail].isend_cnt;
            }

            q.push(next);
        }
    }
}

void SolveProblem() { // nx_node[][] must be sized for indices 1..26
    root = ++node_cnt;
    for(int j=1; j<=node_cnt; j++) for(int k=1; k<=26; k++) {
        int cur = j;
        while(cur != root && !nodes[cur].child.count(k)) cur = nodes[cur].fail;
        if(nodes[cur].child.count(k)) {
            cur = nodes[cur].child[k];
            nx_node[j][k] = cur;
        }
        else nx_node[j][k] = root;
    }
}
```

8.3 Hashing (Rabin Karp)

```
const ll MOD1 = 1e9 + 7, MOD2 = 998244353, p1 = 19192734, p2 = 603820;
const ll inv_p1 = 90700002, inv_p2 = 752581546;
ll pw1[500005], pw2[500005], h1[500005], h2[500005];

void precompute_pow() {
    pw1[0] = pw2[0] = 1;
    for(int i=1; i<500005; i++) { pw1[i] = pw1[i-1] * p1 % MOD1; pw2[i] = pw2[i-1] *
    p2 % MOD2; }
}
void build(string &s) {
    int n = (int) s.size()-1;
    h1[0] = h2[0] = 0;
    for(int i=1; i<=n; i++) { h1[i] = (h1[i-1] * p1 + s[i]) % MOD1; h2[i] = (h2[i-1]
    * p2 + s[i]) % MOD2; }
}
pll get_hash(int l, int r) {
    return {((h1[r] - h1[l-1] * pw1[r-l+1]) % MOD1 + MOD1) % MOD1, ((h2[r] - h2[l-1]
    * pw2[r-l+1]) % MOD2 + MOD2) % MOD2};
}

struct pair_hash {
    template <class T1, class T2>
```

```
    std::size_t operator () (const std::pair<T1,T2> &p) const { return
    std::hash<T1>{}(p.first) ^ std::hash<T2>{}(p.second); }
};
```

8.4 Manacher

```
string arr = " ";
arr += '#';
for(int i=1; i<=n; i++) {
    arr += s[i];
    arr += '#';
}

int r = 0, j = 0;
for (int i=1; i<=n*2+1; i++) {
    if (r < i) dp[i] = 0;
    else {
        int i_prime = 2*j - i;
        dp[i] = min (r-i, dp[i_prime]);
    }
    while (i-dp[i]-1 >= 1 && i+dp[i]+1 <= n*2+1 && arr[i-dp[i]-1] == arr[i+dp[i]+1])
    dp[i]++;
    if (dp[i] + i > r) { r = dp[i] + i; j = i; }
}
```

8.5 Suffix Array + LCP ($O(n \log^2 n)$)

```
string s;
int n;
int groupnum[100005], nx[100005], sa[100005];
int ord[100005], lcp[100005];

ll result = 0;

int len = 1;
bool cmp_sa(int x, int y) {
    if(groupnum[x] == groupnum[y]) {
        if(x+len > n && y+len > n) return true;
        else if(x+len > n) return true;
        else if(y+len > n) return false;
        else return groupnum[x+len] < groupnum[y+len];
    }
    return groupnum[x] < groupnum[y];
}

void suffix_array() {
    for(int i=1; i<=n; i++) sa[i] = i;
    for(int i=1; i<=n; i++) groupnum[i] = s[i] - 'a';
    len = 1;
    while(len <= n) {
        for(int i=1; i<=n; i++) nx[i] = 0;
        sort(sa+1, sa+n+1, cmp_sa);
        for(int i=2; i<=n; i++) nx[sa[i]] = nx[sa[i-1]] + cmp_sa(sa[i-1], sa[i]);
        for(int i=1; i<=n; i++) groupnum[i] = nx[i];
        len = len * 2;
    }
}

void find_lcp() {
    for(int i=1; i<=n; i++) ord[sa[i]] = i;

    int l=0;
    for(int i=1; i<=n; i++) {
        int k = ord[i];
        if(k > 1) {
            int j = sa[k-1];
            while(s[i+l] == s[j+l]) l++;
            lcp[k] = l;
            if(l) l--;
        }
    }
}
```

8.6 Suffix Automaton

```
// Time: O(n) construction over a fixed alphabet (O(1) per char via the ch[26] array;
// swap in a map<int,int> for a large/sparse alphabet).
// 0 = null; node counter starts at 0, root/initial state = node 1
// (allocated via ++node_cnt).
struct sam_node { int len, link, ch[26]; };
sam_node sam[1000005]; int node_cnt = 0, root, last; // size = 2 * max string length

int sam_new_node() {
    int id = ++node_cnt;
    sam[id].len = sam[id].link = 0;
    fill(sam[id].ch, sam[id].ch+26, 0);
    return id;
}

void sam_init() {
    node_cnt = 0;
    root = last = sam_new_node();
}

// c = 0-indexed character id (e.g. s[i]-'a')
void sam_extend(int c) {
    int cur = sam_new_node();
    sam[cur].len = sam[last].len + 1;
    int p = last;
    while(p && !sam[p].ch[c]) {
        sam[p].ch[c] = cur;
        p = sam[p].link;
    }
    if(!p) sam[cur].link = root;
    else {
        int q = sam[p].ch[c];
        if(sam[p].len + 1 == sam[q].len) sam[cur].link = q;
        else {
            int clone = sam_new_node();
            sam[clone] = sam[q];
            sam[clone].len = sam[p].len + 1;
            while(p && sam[p].ch[c] == q) {
                sam[p].ch[c] = clone;
                p = sam[p].link;
            }
            sam[q].link = sam[cur].link = clone;
        }
    }
    last = cur;
}

// usage: sam_init(); for each char c of s (in order), sam_extend(c - 'a');
// each state represents an equivalence class of substrings ending at the
// same set of positions;
// number of distinct substrings of s = sum over all non-root states of
// (len[state] - len[link[state]])
```

8.7 Z Algorithm

```
void calc_z() {
    z[1] = n;
    int l = 1, r = 1;
    for(int i=2; i<=n; i++) {
        z[i] = i < r ? min(r-i, z[i-l+1]) : 0;
        while(i+z[i] <= n && s1[1+z[i]] == s1[i+z[i]]) z[i]++;
        if(i+z[i] > r) { l = i; r = i+z[i]; }
    }
}

void SolveProblem() {
    cin >> s1;
    n = s1.size(); s1 = " " + s1;
    calc_z();
}
```

8.8 LCS - Hirschberg ($O(nm)$)

```
// O(n+m) space
string a, b;
int n, m;

int dp1[7005], dp2[7005];
string result = "";

void DnC(int l, int r, int s, int e) {
    if(l == r) {
        for(int i=s; i<=e; i++) if(a[l] == b[i]) {
            result += a[l];
            break;
        }
        return;
    }

    int mid = (l+r)/2;

    for(int j=s-1; j<=e; j++) dp1[j] = 0;
    for(int i=l; i<=mid; i++) {
        for(int j=e; j>=s; j--) {
            if(a[i] == b[j]) dp1[j] = max(dp1[j], dp1[j-1] + 1);
        }
        for(int j=s+1; j<=e; j++) dp1[j] = max(dp1[j], dp1[j-1]);
    }

    for(int j=s; j<=e+1; j++) dp2[j] = 0;
    for(int i=r; i>=mid+1; i--) {
        for(int j=s; j<=e; j++) {
            if(a[i] == b[j]) dp2[j] = max(dp2[j], dp2[j+1] + 1);
        }
        for(int j=e-1; j>=s; j--) dp2[j] = max(dp2[j], dp2[j+1]);
    }

    int nx = -1, mx = -1;
    for(int i=s-1; i<=e; i++) {
        if(mx < dp1[i] + dp2[i+1]) {
            mx = dp1[i] + dp2[i+1];
            nx = i;
        }
    }

    DnC(l, mid, s, nx); DnC(mid+1, r, nx+1, e);
}

void SolveProblem() {
    cin >> a >> b;
    n = a.size(); a = " " + a;
    m = b.size(); b = " " + b;

    DnC(1, n, 1, m);

    cout << result.size() << "\n";
    cout << result << "\n";
}
```

9 Miscellaneous

9.1 Random Generator

```
random_device rd;
mt19937 gen(rd());
cout << gen() << " "; // raw output of the generator. 0~2^32-1
uniform_int_distribution<> dist(1,6); // distribute results between 1 and 6
inclusive.
cout << dist(gen) << " "; // pass the generator to the distribution.
shuffle(all(arr), gen);
```

9.2 Rope

```
#include <ext/rope>
using namespace __gnu_cxx;

crope r(s.c_str());
crope s1 = r.substr(x, l); // x ~ x+l-1의 substring
r.erase(x, l); // x ~ x+l-1 까지 삭제
r = s1 + r; // s1와 r 더하기
```

9.3 PBDS

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
#define ordered_set tree<pll, null_type, less<pll>,
rb_tree_tag,tree_order_statistics_node_update>
```

(Set 이름).order_of_key(NUM) : ordered_set 에서 NUM 보다 작은(미만의) 원소의 개수를 반환한다.
(Set 이름).find_by_order(K) : ordered_set 에서 (K+1)번째 원소가 있는 iterator 을 반환한다. (K가 0이면 1번째)

9.4 Big Integer

```
#include <boost/multiprecision/cpp_int.hpp>
using namespace boost::multiprecision;
// 큰 상수는 문자열로 초기화
cpp_int a = 1, b("123456789012345678901234567890");
cpp_int c = a + b, d = b * b, e = d / b, f = d % b; // 사칙연산 지원
cpp_int g = pow(b, 10); // 거듭제곱
cout << b << "\n"; // cin/cout 그대로 사용 가능
```

9.5 Mo's

```
struct query {
    int start, end, num;
    bool operator< (const query& other) const {
        if(start / sqrtN == other.start / sqrtN) return end < other.end;
        return start / sqrtN < other.start / sqrtN;
    }
};
ll cur_ans = 0;
void add(int idx) { /* fill in per-problem */ }
void remove(int idx) { /* fill in per-problem */ }
void solve_mo(vector<query> &queries, vl &ans) {
    sort(all(queries));
    int cur_l = 1, cur_r = 0;
    for(query &q : queries) {
        while(cur_r < q.end) add(++cur_r);
        while(cur_l > q.start) add(--cur_l);
        while(cur_r > q.end) remove(cur_r--);
        while(cur_l < q.start) remove(cur_l++);
        ans[q.num] = cur_ans;
    }
}
```

9.6 CDQ ($O(n \log n)$)

```
ll cdq(int l, int r) {
    if(l == r) return 0;

    int mid = (l+r)/2;
    ll curresult = cdq(l, mid) + cdq(mid+1, r);

    int idx_l = l, idx_r = mid+1, idx_tmp = l;
    while(idx_l <= mid && idx_r <= r) {
        if(idx[idx_l][2] <= idx[idx_r][2]) {
            S.update(idx[idx_l][3], 1);

            for(int j=1; j<=3; j++) tmp[idx_tmp][j] = idx[idx_l][j];
            idx_tmp++; idx_l++;
        }
        else {
            curresult += S.find_sum(idx[idx_r][3]);

            for(int j=1; j<=3; j++) tmp[idx_tmp][j] = idx[idx_r][j];
            idx_tmp++; idx_r++;
        }
    }

    int idx_l1 = idx_l, idx_r1 = idx_r;
    while(idx_l1 <= mid) {
        for(int j=1; j<=3; j++) tmp[idx_tmp][j] = idx[idx_l1][j];
        idx_tmp++; idx_l1++;
    }
    while(idx_r1 <= r) {
        curresult += S.find_sum(idx[idx_r1][3]);

        for(int j=1; j<=3; j++) tmp[idx_tmp][j] = idx[idx_r1][j];
        idx_tmp++; idx_r1++;
    }

    for(int i=l; i<idx_l; i++) S.update(idx[i][3], -1);
    for(int i=l; i<=r; i++) for(int j=1; j<=3; j++) idx[i][j] = tmp[i][j];

    return curresult;
}
```

9.7 Faker's Algorithm ($O(n \log n)$)

```
// Faker's Algorithm: n jobs form a tree rooted at 'root' (a job can only
// run after its parent has run), each job i with weight arr[i]. Among all
// orderings that respect this precedence, minimizes sum_i arr[i] *
// (completion time of i), where completion time = how many jobs have run
// up to and including i (unit processing time per job). Generalizes
// Smith's rule (for a single chain with no precedence, schedule by
// decreasing weight) to tree precedence: repeatedly take the available
// group with the largest average weight (sum/size) and merge it right
// after its parent's current group, via a lazy priority queue over
// DSU-merged groups.
struct fraction { // a / b
    ll a, b;
    fraction() { a = 0; b = 1;}
    fraction(ll _a) : a(_a) { b = 1; }
    fraction(ll _a, ll _b) : a(_a), b(_b) { }

    friend bool operator < (const fraction &f1, const fraction &f2) {
        return f1.a * f2.b < f2.a * f1.b;
    }
    friend bool operator > (const fraction &f1, const fraction &f2) {
        return f1.a * f2.b > f2.a * f1.b;
    }
    friend bool operator != (const fraction &f1, const fraction &f2) {
        return f1.a * f2.b != f2.a * f1.b;
    }
};

int n, root;
ll arr[1005];
vi edges[1005];

int parent[1005];
void dfs(int index, int p) {
    parent[index] = p;
    for(int i : edges[index]) if(i != p) dfs(i, index);
}

int group[1005], sz[1005];
ll sumarr[1005] = {0};
int find_group(int index) {
    return group[index] == index ? index : group[index] = find_group(group[index]);
}

ll result = 0;

void SolveProblem() {
    cin >> n >> root;
    for(int i=1; i<=n; i++) cin >> arr[i];
    for(int i=1; i<=n-1; i++) {
        int a, b; cin >> a >> b;
        edges[a].push_back(b); edges[b].push_back(a);
    }

    dfs(root, -1);

    for(int i=1; i<=n; i++) {
        group[i] = i;
        sz[i] = 1;
        sumarr[i] = arr[i];
    }

    priority_queue<pair<fraction, int>> pq;
    for(int i=1; i<=n; i++) pq.push({fraction(sumarr[i], sz[i]), i});

    for(int i=1; i<=n; i++) result += arr[i];

    while(!pq.empty()) {
        fraction f; int cur;
        tie(f, cur) = pq.top(); pq.pop();

        if(f.b != sz[cur] || cur == root) continue;

        int g = find_group(parent[cur]);
        result += sumarr[cur] * sz[g];

        group[cur] = g;
        sz[g] += sz[cur]; sumarr[g] += sumarr[cur];

        pq.push({fraction(sumarr[g], sz[g]), g});
    }

    cout << result << "\n";
}
```

9.8 Matroid Intersection

```
// Time: O(r^2 n) independence-oracle calls (r = |result|, n = |E|). Finds
// a maximum common independent set of two matroids on ground set
// {0,...,n-1}, given independence oracles indep1/indep2 (each takes a
// candidate subset as a vector<bool> of size n, true iff independent).
vector<int> matroid_intersection(int n, function<bool(vector<bool>&)> indep1,
function<bool(vector<bool>&)> indep2) {
    vector<bool> in_set(n, false);
    while(true) {
        vector<int> par(n, -2);
        queue<int> q;
        int sink = -1;
        for(int y=0; y<n && sink==-1; y++) if(!in_set[y]) {
            in_set[y] = true; bool a1 = indep1(in_set); in_set[y] = false;
            if(a1) {
                par[y] = -1;
                q.push(y);
                in_set[y] = true; bool a2 = indep2(in_set); in_set[y] = false;
                if(a2) sink = y;
            }
        }
        while(!q.empty() && sink==-1) {
            int u = q.front(); q.pop();
            for(int v=0; v<n; v++) if(par[v]==-2) {
                bool edge = false;
                if(in_set[u] && !in_set[v]) {
                    in_set[u] = false; in_set[v] = true;
                    edge = indep1(in_set);
                    in_set[u] = true; in_set[v] = false;
                } else if(!in_set[u] && in_set[v]) {
                    in_set[v] = false; in_set[u] = true;
                    edge = indep2(in_set);
                    in_set[v] = true; in_set[u] = false;
                }
                if(edge) {
                    par[v] = u;
                    if(!in_set[v]) {
                        in_set[v] = true; bool a2 = indep2(in_set); in_set[v] =
                        false;
                        if(a2) { sink = v; break; }
                    }
                    q.push(v);
                }
            }
        }
        if(sink==-1) break;
        int cur = sink;
        while(cur != -1) { in_set[cur] = !in_set[cur]; cur = par[cur]; }
    }
    vector<int> result;
    for(int i=0; i<n; i++) if(in_set[i]) result.push_back(i);
    return result;
}
```

9.9 Primes and Divisors Number

< 10^k	number	divisors	2	3	5	71113171923293137
1	6	4	1	1		
2	60	12	2	1	1	
3	840	32	3	1	1	1
4	7560	64	3	3	1	1
5	83160	128	3	3	1	1 1
6	720720	240	4	2	1	1 1 1
7	8648640	448	6	3	1	1 1 1
8	73513440	768	5	3	1	1 1 1 1
9	735134400	1344	6	3	2	1 1 1 1
10	6983776800	2304	5	3	2	1 1 1 1 1
11	97772875200	4032	6	3	2	2 1 1 1 1
12	963761198400	6720	6	4	2	1 1 1 1 1 1
13	9316358251200	10752	6	3	2	1 1 1 1 1 1 1
14	97821761637600	17280	5	4	2	2 1 1 1 1 1 1
15	866421317361600	26880	6	4	2	1 1 1 1 1 1 1 1
16	8086598962041600	41472	8	3	2	2 1 1 1 1 1 1 1
17	74801040398884800	64512	6	3	2	2 1 1 1 1 1 1 1 1
18	897612484786617600	103680	8	4	2	2 1 1 1 1 1 1 1 1

< 10^k	prime	# of prime	< 10^k	prime
1	7	4	10	9999999967
2	97	25	11	99999999977
3	997	168	12	999999999989
4	9973	1229	13	9999999999971
5	99991	9592	14	99999999999973
6	999983	78498	15	999999999999989
7	9999991	664579	16	9999999999999937
8	99999989	5761455	17	99999999999999997
9	999999937	50847534	18	999999999999999989

9.10 Lemma

- **Lucas's Theorem**

$$\binom{n}{k} \equiv \prod_{i=0}^{r} \binom{n_i}{k_i} \pmod{p}$$

where p is the prime and $n = n_m n_{m-1} \cdots n_0$ and $k = k_m k_{m-1} \cdots k_0$ are the base representations of p.

- **Euler's Theorem**

$$a^{\varphi(n)} \equiv 1 \pmod{n} \quad \text{for } \gcd(a,n) = 1$$

- **Möbius Inversion Formula**

$$f(n) = \sum_{d|n} g(d) \iff g(n) = \sum_{d|n} \mu(d)\, f\left(\frac{n}{d}\right)$$

$$\sum_{d|n} \phi(d) = n \iff \phi(n) = \sum_{d|n} \mu(d)\, \frac{n}{d}$$

- **Green's Theorem**

$$\oint_C (P\,dx + Q\,dy) = \iint_D \left(\frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y}\right) dA$$

- **Fermat's Little Theorem**

$$a^{p-1} \equiv 1 \pmod{p} \quad \text{for } \gcd(a,p) = 1$$

Equivalently, $a^p \equiv a \pmod{p}$ for all integers a.

- **Pick's theorem**

$$A = I + \frac{B}{2} - 1$$

where A is the area of a simple polygon whose vertices are points on a lattice. I is the number of interior lattice points, and B is the number of boundary lattice points.

- **Burnside's Lemma**

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

where X is a set, G is a group acting on X. X/G is the set of orbits, and X^g is the set of elements fixed by g.

- **Kirchhoff's Theorem**

$$\text{Number of spanning trees } = \det(L^*)$$

where L is the Laplacian matrix of the graph and L^* is any cofactor of L.

- **Cayley's Formula**

$$\text{Number of labeled trees with } n \text{ vertices} = n^{n-2}$$

- **Simpson's Rule**

$$\int_a^b f(x)\,dx \approx \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right]$$

- **Law of Sines**

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C}$$

- **Law of Cosines**

$$c^2 = a^2 + b^2 - 2ab\cos C$$

- **Area (Heron's Formula)**

$$A = \sqrt{s(s-a)(s-b)(s-c)} \quad \text{where } s = \frac{a+b+c}{2}$$

- **Angle Addition Formulas**

$$\sin(A \pm B) = \sin A \cos B \pm \cos A \sin B$$
$$\cos(A \pm B) = \cos A \cos B \mp \sin A \sin B$$
$$\tan(A \pm B) = \frac{\tan A \pm \tan B}{1 \mp \tan A \tan B}$$

- **Double Angle Formulas**

$$\sin(2x) = 2\sin x \cos x$$
$$\cos(2x) = \cos^2 x - \sin^2 x = 2\cos^2 x - 1 = 1 - 2\sin^2 x$$
$$\tan(2x) = \frac{2\tan x}{1 - \tan^2 x}$$

- **Half Angle Formulas**

$$\sin\left(\frac{x}{2}\right) = \pm\sqrt{\frac{1-\cos x}{2}}, \quad \cos\left(\frac{x}{2}\right) = \pm\sqrt{\frac{1+\cos x}{2}}$$
$$\tan\left(\frac{x}{2}\right) = \frac{\sin x}{1+\cos x}$$

- **Catalan Numbers**

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$
$$C_0 = 1, \quad C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i} \quad \text{for } n \geq 0$$

$1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786$

- **Ballot Number**

$$B(p,q) = \frac{p-q}{p+q}\binom{p+q}{p} \quad (p > q \geq 0)$$

counts vote sequences where A stays strictly ahead throughout; $B(n+1, n) = C_n$.

- **Motzkin Numbers**

$$M_0 = 1, \quad M_{n+1} = M_n + \sum_{k=0}^{n-1} M_k M_{n-1-k}$$

counts paths $(0,0) \to (n,0)$ with steps $U = (1,1), D = (1,-1), F = (1,0)$ that never go below the x-axis.

- **Narayana Numbers**

$$N(n,k) = \frac{1}{n}\binom{n}{k}\binom{n}{k-1} \quad (1 \leq k \leq n)$$

counts Dyck paths of length $2n$ with exactly k peaks; $\sum_{k=1}^{n} N(n,k) = C_n$.

- **Schröder Numbers (large)**

$$S_0 = 1,\ S_1 = 2,\ (n+1)S_n = 3(2n-1)S_{n-1} - (n-2)S_{n-2} \quad (n \geq 2)$$

counts monotone paths $(0,0) \to (n,n)$ with steps $(1,0), (0,1), (1,1)$ that never rise above $y = x$.

- **Stirling Numbers (1st kind, unsigned)**

$$\begin{bmatrix} n \\ k \end{bmatrix} = (n-1)\begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix}, \quad \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 1$$

unsigned $\begin{bmatrix} n \\ k \end{bmatrix}$ counts permutations of n elements with k cycles.

- **Stirling Numbers (2nd kind)**

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = k\begin{Bmatrix} n-1 \\ k \end{Bmatrix} + \begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix} = \frac{1}{k!}\sum_{j=0}^{k}(-1)^j\binom{k}{j}(k-j)^n$$

counts partitions of an n-set into k nonempty unlabeled blocks.

- **Eulerian Numbers**

$$A(n,k) = (k+1)A(n-1,k) + (n-k)A(n-1,k-1), \quad A(1,0) = 1$$

counts permutations of $1..n$ with exactly k descents ($0 \le k \le n-1$).

- **Bell Numbers**

$$B_0 = 1, \quad B_{n+1} = \sum_{k=0}^{n}\binom{n}{k}B_k = \sum_{k=0}^{n}\begin{Bmatrix} n \\ k \end{Bmatrix}$$

counts partitions of an n-set into any number of nonempty unlabeled blocks. Equivalently, $B_n = n!\cdot[x^n]\,e^{e^x-1}$: expand e^{e^x-1} as a formal power series and take $n!$ times the coefficient of x^n.

- **Partition Numbers**

$$p(n) = \sum_{k=1}^{\infty}(-1)^{k-1}\left[p\Big(n-\tfrac{k(3k-1)}{2}\Big) + p\Big(n-\tfrac{k(3k+1)}{2}\Big)\right], \; p(0) = 1$$

(pentagonal number theorem) counts ways to write n as a sum of positive integers, order not mattering.

- **Parking Functions**

$$\text{count of length-}n\text{ parking functions} = (n+1)^{n-1}$$

(a_i), $1 \le a_i \le n$, is one iff sorted $b_1 \le \cdots \le b_n$ has $b_i \le i$ for all i.

- **Fibonacci Numbers**

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad \text{for } n \ge 2$$

$$\sum_{i=0}^{n} F_i = F_{n+2} - 1$$

$$\sum_{i=0}^{n} F_i^2 = F_n F_{n+1}$$

$$F_n^2 - F_{n-1}F_{n+1} = (-1)^{n+1}$$

$$\gcd(F_m, F_n) = F_{\gcd(m,n)}$$

$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987$

- **Point and Line Distance**

$$d = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$$

- **Derangement**

$$!n = n!\sum_{k=0}^{n}\frac{(-1)^k}{k!}$$

$!n$ = permutations of n elements with no fixed points.

- **Harmonic Series**

$$H_n = \sum_{k=1}^{n}\frac{1}{k} \approx \ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + \frac{1}{120n^4}$$

where $\gamma \approx 0.5772156649$ is the Euler-Mascheroni constant.

- **Stirling's Approximation**

$$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

- **Taylor Series**

$$e^x = \sum_{n=0}^{\infty}\frac{x^n}{n!}, \quad \sin x = \sum_{n=0}^{\infty}(-1)^n\frac{x^{2n+1}}{(2n+1)!}$$

$$\cos x = \sum_{n=0}^{\infty}(-1)^n\frac{x^{2n}}{(2n)!}, \quad \ln(1+x) = \sum_{n=1}^{\infty}(-1)^{n+1}\frac{x^n}{n}$$

- **Lifting the exponent lemma**

$$v_p(a^n - b^n) = v_p(a-b) + v_p(n) \quad (p \mid a-b,\ p \nmid ab)$$
$$v_p(a^n - b^n) = v_p(a-b) + v_p(a+b) + v_p(n) - 1 \quad (p = 2,\ n \text{ even})$$

- **Matroid**
A matroid $M = (E, \mathcal{I})$ is a ground set E with a family of "independent" subsets $\mathcal{I} \subseteq 2^E$ satisfying: (I1) $\emptyset \in \mathcal{I}$; (I2) hereditary: $B \in \mathcal{I}, A \subseteq B \implies A \in \mathcal{I}$; (I3) exchange: $A, B \in \mathcal{I}, |A| < |B| \implies \exists x \in B \setminus A$ with $A \cup \{x\} \in \mathcal{I}$.
A maximal independent set is a basis; by (I3) all bases have the same size, the rank $r(M)$. The greedy algorithm (sort elements by weight, add each if it keeps the current set independent) finds a max-weight independent set / basis for any matroid, and only for matroids (this is essentially matroid's defining property for optimization). Common examples: graphic matroid (E = edges, independent = forests – greedy on this is Kruskal's MST), linear matroid (E = vectors, independent = linearly independent sets), uniform matroid $U_{k,n}$ (independent = any subset of size $\le k$), transversal matroid of a bipartite graph $G = (X, Y, E)$ ($A \subseteq X$ independent iff matchable into Y – see Matching is a Linear Matroid for its linear representation).

- **Matroid Intersection Theorem**
For two matroids $M_1 = (E, \mathcal{I}_1)$, $M_2 = (E, \mathcal{I}_2)$ on the same ground set,

$$\max_{I \in \mathcal{I}_1 \cap \mathcal{I}_2} |I| = \min_{A \subseteq E}\big(r_1(A) + r_2(E \setminus A)\big)$$

where r_1, r_2 are the rank functions of M_1, M_2. A maximum common independent set can be found in polynomial time by repeatedly augmenting along a shortest path in the exchange graph (arcs $y \to x$ for $x \in I, y \notin I$ if $I - x + y \in \mathcal{I}_1$, and $x \to y$ if $I - x + y \in \mathcal{I}_2$, plus source arcs from $\{y : I + y \in \mathcal{I}_1\}$ and sink arcs to $\{y : I + y \in \mathcal{I}_2\}$), analogous to augmenting paths in bipartite matching. Bipartite matching itself is the intersection of two partition (transversal) matroids on the edge set, one per side; this is why matching, arborescence, and colored-forest problems all reduce to matroid intersection. (Three or more matroids: intersection is NP-hard in general.)

10 Template

```cpp
#include <bits/stdc++.h>
using namespace std;

#pragma GCC optimize("O3")
#pragma GCC optimize("Ofast")
#pragma GCC optimize("unroll-loops")

typedef long long ll;
typedef long double ld;
typedef __int128 i128;
typedef complex<long double> cpx;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<ld> vld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef pair<ld, ld> pld;

#define prec(x)    \
    cout << fixed; \
    cout.precision(x);

#define all(v) v.begin(), v.end()

// const ll MOD = 1e9+7;

// int dx[9] = {0, -1, 0, 1, 0, -1, -1, 1, 1};
// int dy[9] = {0, 0, 1, 0, -1, -1, 1, 1, -1};

void SolveProblem() {

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    int test_case = 1;
    // cin >> test_case;
    for(int tc = 1; tc <= test_case; tc++) {
        SolveProblem();
    }
}
```